



Specifica Tecnica

The Bitless (Gruppo 18):

Lorenzo Battistella (2103116), Edoardo De Piccoli (2101055), Davide Facco (2087852),
Anna Marini (2110986), Andrea Menegaldo (2116426), Samuele Vendramin (2111934),
Simone Zecchinato (2113189)

Informazioni documento			
Versione	Data	Stato	Destinatari
0.8.1	2026-08-13	Stesura	The Bitless, prof. T. Vardanega, prof. R. Cardin, Zucchetti

Contatti: thebitless.swe@gmail.com

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.8.1	2026-08-13	E. De Piccoli		-	Revisione delle sezioni Architettura del backend, Idiomi e convenzioni di codifica e dei relativi flussi: correzioni dei riferimenti ai casi d'uso, condensazione del testo e riordino delle figure
0.8.0	2026-08-13	E. De Piccoli		-	Stesura dei flussi di generazione a partire da un collegamento e di propagazione degli errori
0.7.0	2026-08-13	E. De Piccoli		-	Stesura di Idiomi e convenzioni di codifica
0.6.0	2026-08-13	E. De Piccoli		-	Stesura dell'Architettura del backend
0.5.0	2026-08-11	A. Marini		-	Allineamento della sezione Tecnologie al repository di prodotto, stesura della Panoramica architetturale e riorganizzazione dello scheletro delle sezioni di Architettura
0.4.0	2026-07-27	A. Marini	A. Menegaldo	-	Integrazione continua e analisi statica del backend, tracciamento dei requisiti nella sezione Tecnologie, riorganizzazione della struttura delle sezioni Architettura, Flusso dei dati e Tracciamento
0.3.0	2026-07-27	A. Marini	A. Menegaldo	-	Stesura della sezione Tecnologie
0.2.0	2026-07-27	A. Marini	A. Menegaldo	-	Riorganizzazione della struttura del documento e stesura della sezione Introduzione
0.1.0	2026-07-18	A. Menegaldo	A. Marini	-	Struttura del documento #1

Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Scopo del prodotto	6
1.3	Glossario	7
1.4	Riferimenti	7
1.4.1	Riferimenti normativi	7
1.4.2	Riferimenti informativi	7
2	Tecnologie	10
2.1	Criteri di scelta	10
2.2	Frontend	11
2.3	Backend	15
2.4	Persistenza delle note	18
2.5	Tecnologie di test e analisi	21
2.5.1	Analisi statica	21
2.5.2	Analisi dinamica	22
2.6	Infrastruttura	24
3	Architettura	28
3.1	Panoramica architetturale	28
3.1.1	Visione d'insieme	28
3.1.2	Stile architetturale del servizio applicativo	29
3.1.3	Stile architetturale dell'applicazione web	30
3.1.4	Vocabolario	30
3.2	Architettura di deployment	31
3.2.1	Unità di esecuzione e configurazione	31
3.2.2	Confronto con l'architettura a microservizi	31
3.2.3	Confronto con il monolite a strati	31
3.3	Architettura del frontend	31
3.3.1	Organizzazione dei sorgenti	31
3.3.2	Scomposizione in componenti	31
3.3.3	Gestione dello stato applicativo	31
3.3.4	Integrazione dell'editor	31
3.3.5	Accesso al servizio applicativo	31
3.3.6	Persistenza lato client	31
3.4	Architettura del backend	31
3.4.1	Struttura a esagono e contratti di dipendenza	32
3.4.2	Adattatori primari	33
3.4.3	Il nucleo	35
3.4.4	Adattatori secondari	37
3.4.5	Composition root e configurazione	39
3.5	Design pattern adottati	40
3.5.1	Object Adapter	40
3.5.2	Facade	40
3.5.3	Observer	40
3.5.4	Dependency Injection	40
3.5.5	Pattern valutati e non adottati	40

3.6	Idiomi e convenzioni di codifica	40
3.6.1	Generatori asincroni e propagazione dell'annullamento	41
3.6.2	Tipizzazione dei confini fra i livelli	42
3.6.3	Denominazione, struttura dei file e gestione degli errori	43
4	Flusso dei dati	44
4.1	Generazione assistita da LLM_G con risposta in streaming	44
4.2	Annullamento di un'operazione in corso	44
4.3	Generazione a partire da un collegamento	44
4.4	Salvataggio e caricamento di una nota	46
4.5	Propagazione e presentazione degli errori	46
5	Tracciamento dei requisiti$_G$	48
5.1	Criteri di tracciamento	48
5.2	Requisiti $_G$ funzionali	48
5.3	Requisiti $_G$ di qualità	48
5.4	Requisiti $_G$ di vincolo $_G$	48
5.5	Grafici riassuntivi	48

Elenco delle tabelle

1	Tecnologie adottate per il frontend	12
2	Tecnologie adottate per il backend	15
3	Strumenti di analisi statica	21
4	Strumenti di analisi dinamica	23
5	Tecnologie di infrastruttura	24
6	Vocabolario architetturale	30
7	Contratti di dipendenza dichiarati in <code>api/.importlinter</code>	33

Elenco delle figure

1	Architettura del servizio applicativo	32
---	---	----

1 Introduzione

1.1 Scopo del documento

Il presente documento descrive l'architettura del sistema_G **Second Brain**, prodotto realizzato dal gruppo **The Bitless** in risposta al capitolato_G **C6** proposto da **Zucchetti S.p.A.**. Mentre l'Analisi dei Requisiti_G stabilisce *che cosa* il sistema_G deve fare, la Specifica Tecnica stabilisce *come* il sistema_G è costruito per farlo.

Nello specifico, il documento persegue i seguenti obiettivi:

- **Motivazione delle scelte tecnologiche:** espone le tecnologie adottate per ciascun livello del prodotto, i bisogni tecnici che ne hanno determinato la selezione e le alternative valutate e scartate, con le relative motivazioni.
- **Descrizione dell'architettura:** illustra la scomposizione del sistema_G nelle sue componenti, le responsabilità attribuite a ciascuna di esse e le interazioni che ne regolano la collaborazione, sia in termini logici sia in termini di distribuzione delle parti in esecuzione.
- **Documentazione dei design pattern:** riporta i pattern architetturali e di progettazione adottati, motivandone l'impiego rispetto al problema specifico che risolvono all'interno del prodotto.
- **Tracciamento dei requisiti_G:** correla le componenti architetturali ai requisiti_G individuati nell'Analisi dei Requisiti_G, così da rendere verificabile la copertura del perimetro funzionale concordato con la proponente_G.

Il documento costituisce il riferimento progettuale per le successive fasi di realizzazione e di manutenzione del prodotto: ogni intervento sul codice deve risultare coerente con quanto qui descritto e, qualora introduca una variazione architetturale, deve essere preceduto dall'aggiornamento della presente specifica.

La Specifica Tecnica dà inoltre continuità al **Proof of Concept_G** sviluppato per la **Requirements and Technology Baseline_G**: le tecnologie e le soluzioni architetturali qui documentate sono quelle sperimentate e validate in quella sede, ora consolidate e portate al livello di dettaglio necessario allo sviluppo_G del prodotto completo.

Come gli altri documenti del gruppo, la stesura segue un approccio **incrementale_G**: il documento viene raffinato a ogni iterazione, in modo da riflettere costantemente lo stato effettivo del prodotto.

1.2 Scopo del prodotto

Il capitolato_G **C6** di **Zucchetti S.p.A.** richiede la realizzazione di **Second Brain**, un'applicazione web per la scrittura e la gestione di note in formato **Markdown_G**, assistita da modelli linguistici di grandi dimensioni (**LLM_G**).

L'interfaccia si articola in due pannelli affiancati: a sinistra un editor in cui l'utente compone il testo nella sintassi **Markdown_G**, a destra un'anteprima che ne mostra il rendering aggiornato in tempo reale. Su questa base si innestano le funzioni assistite dall'**LLM_G**, che operano sul testo selezionato o sull'intera nota: riassunto, riscrittura e adattamento del tono, correzione, traduzione multilingue, analisi critica secondo la tecnica dei **Sei Cappelli per Pensare_G** di Edward de Bono e generazione di testo secondo il paradigma del **Distant Writing_G**, in cui l'utente descrive sinteticamente il contenuto desiderato e delega al modello la stesura. Le note prodotte sono salvate e ricaricate sul file system locale dell'utente in formato **Markdown_G**.

Il valore del prodotto non risiede quindi nella sola redazione del testo, ma nel ruolo attivo assunto dal sistema_G, che affianca l'utente nelle fasi di ideazione, revisione e riorganizzazione dei contenuti.

Per la trattazione completa del perimetro funzionale, dei casi d'uso_G e della classificazione dei requisiti_G in obbligatori, desiderabili e opzionali si rimanda all'*Analisi dei Requisiti*, di cui il presente documento non intende costituire una duplicazione.

1.3 Glossario

La creazione e lo sviluppo_G di un sistema_G software richiedono una complessa operazione di progettazione e analisi del dominio_G, attività che deve necessariamente precedere la stesura del codice. Il gruppo si impegna a raccogliere e organizzare tali informazioni in modo che siano facilmente accessibili, al fine di favorire la massima asincronia ed efficienza nelle attività di progetto.

La principale fonte di ambiguità nello svolgimento delle attività è rappresentata dall'incomprensione dei termini tecnici o specialistici adottati. A tale scopo, la nomenclatura di riferimento viene raccolta nel *glossario*, un documento incrementale_G volto a definire ogni termine rilevante per il dominio_G del progetto.

Il gruppo si impegna a contrassegnare ogni occorrenza di un termine presente nel glossario mediante l'apposizione di una lettera **G** a pedice, come esemplificato di seguito:
parola_G

Al fine di garantire una corretta comprensione del dominio_G, è fondamentale che ogni membro del gruppo consulti periodicamente il glossario, assicurando così il costante allineamento sulle definizioni e sui concetti chiave.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato C6 - Second Brain:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
Ultimo accesso: 20 maggio 2026.
- **Regolamento del Progetto Didattico:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Ultimo accesso: 23 maggio 2026.
- **Norme di Progetto v1.0.0:**
https://thebitless.live/RTB/doc_interna/norme-di-progetto/norme-di-progetto.pdf
Ultimo accesso: 8 giugno 2026.

1.4.2 Riferimenti informativi

Documentazione di progetto

- **Glossario v1.0.0:**
https://thebitless.live/RTB/doc_interna/glossario/glossario.pdf
Ultimo accesso: 12 giugno 2026.

- **Analisi dei Requisiti v2.0.0:**
https://thebitless.live/RTB/doc_esterna/analisi-dei-requisiti/analisi-dei-requisiti.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale interno VI_2026-05-12 – Scelta delle tecnologie per il PoC_G:**
https://thebitless.live/RTB/doc_interna/verballi/VI_2026-05-12_quarto-sprint/VI_2026-05-12_quarto-sprint.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale interno VI_2026-05-20 – Progettazione del PoC_G:**
https://thebitless.live/RTB/doc_interna/verballi/VI_2026-05-20_progettazione-PoC/VI_2026-05-20_progettazione-PoC.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale esterno VE_2026-05-14 – Approvazione delle tecnologie da parte della proponente_G:**
https://thebitless.live/RTB/doc_esterna/verballi/VE_2026-05-14_AdR-scelta-tecnologie/VE_2026-05-14_AdR-scelta-tecnologie.pdf
Ultimo accesso: 27 luglio 2026.

Documentazione delle tecnologie adottate Documentazione ufficiale delle tecnologie descritte nella sezione 2. *Ultimo accesso* a tutte le risorse elencate di seguito: 27 luglio 2026.

- **TypeScript:** <https://www.typescriptlang.org/docs/>
- **React:** <https://react.dev/>
- **Zustand:** <https://zustand.docs.pmnd.rs/>
- **react-markdown:** <https://github.com/remarkjs/react-markdown>
- **Tailwind CSS:** <https://tailwindcss.com/docs>
- **shadcn/ui:** <https://ui.shadcn.com/>
- **Radix UI Primitives:** <https://www.radix-ui.com/primitives>
- **CodeMirror 6:** <https://codemirror.net/docs/>
- **Vite:** <https://vite.dev/>
- **Python 3.12:** <https://docs.python.org/3.12/>
- **FastAPI:** <https://fastapi.tiangolo.com/>
- **Pydantic:** <https://docs.pydantic.dev/>
- **httpx:** <https://www.python-httpx.org/>
- **Uvicorn:** <https://www.uvicorn.org/>
- **Tavily:** <https://docs.tavily.com/>
- **LiteLLM:** <https://docs.litellm.ai/>
- **Server-Sent Events (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- **File System Access API (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/File_System_API
- **Specifica OpenAPI:** <https://spec.openapis.org/oas/latest.html>

- **openapi-typescript**: <https://openapi-ts.dev/>
- **ESLint**: <https://eslint.org/docs/latest/>
- **Ruff**: <https://docs.astral.sh/ruff/>
- **mypy**: <https://mypy.readthedocs.io/>
- **Vitest**: <https://vitest.dev/>
- **Copertura del codice in Vitest**: <https://vitest.dev/guide/coverage>
- **Testing Library**: <https://testing-library.com/docs/react-testing-library/intro/>
- **pytest**: <https://docs.pytest.org/>
- **Docker**: <https://docs.docker.com/>
- **Docker Compose**: <https://docs.docker.com/compose/>
- **GitHub Actions**: <https://docs.github.com/actions>
- **pnpm**: <https://pnpm.io/>
- **uv**: <https://docs.astral.sh/uv/>
- **Node.js – calendario di supporto delle versioni**: <https://github.com/nodejs/release>
- **File API – interfaccia File (MDN)**: <https://developer.mozilla.org/en-US/docs/Web/API/File>

Riferimenti metodologici

- **E. Gamma, R. Helm, R. Johnson, J. Vlissides**, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- **C4 model for visualising software architecture**:
<https://c4model.com/>
Ultimo accesso: 27 luglio 2026.

2 Tecnologie

2.1 Criteri di scelta

Le tecnologie descritte in questa sezione non sono state selezionate in astratto, sulla base della sola diffusione o popolarità, ma in risposta a bisogni tecnici precisi derivati dal capitolato_G e dall'Analisi dei Requisiti_G. Ciascuno dei bisogni elencati di seguito vincola una o più delle scelte successive, e costituisce il criterio rispetto al quale tali scelte vanno valutate.

1. Risposta progressiva. Un riassunto o una generazione di testo richiedono al modello linguistico un tempo di elaborazione dell'ordine dei secondi. Mantenere l'utente davanti a un semplice indicatore di attesa per tutta la durata dell'operazione non è accettabile sul piano dell'usabilità_G: il testo deve comparire progressivamente mentre viene prodotto (*streaming*). Questo bisogno determina la scelta di un livello server asincrono, di un client HTTP che supporti la lettura incrementale della risposta e di un protocollo di trasporto adatto al flusso unidirezionale di dati testuali. Il criterio discende dal requisito_G **R-109-F-De**, che impone un indicatore di avanzamento per l'intera durata delle operazioni con attesa percepibile, e da **R-79-F-De**, che richiede la possibilità di annullare un'elaborazione in corso: entrambi presuppongono che il server mantenga il controllo dell'operazione mentre questa procede, e non si limiti a restituirne l'esito finale.

2. Riservatezza delle credenziali dei fornitori esterni. L'accesso ai modelli linguistici avviene tramite una chiave di autenticazione fornita dalla proponente_G, in attuazione del requisito di vincolo_G **R-2-V-Ob**, che prescrive l'interfacciamento con i servizi LLM_G tramite API_G. Tale chiave non deve in alcun caso raggiungere il browser, dove sarebbe leggibile da chiunque ispezioni il traffico o il codice dell'applicazione. È quindi necessaria una componente server intermediaria che detenga la chiave e sia l'unico soggetto a comunicare con il fornitore; la medesima componente risolve anche il vincolo_G **R-3-V-Ob** sulla *same-origin policy*, poiché è essa a esporre al browser un'origine controllata, configurata mediante la politica CORS.

Il criterio si estende a ogni fornitore esterno contattato dal prodotto, non al solo fornitore dei modelli. La funzione di generazione a partire da un collegamento (**R-74-F-De**) richiede l'estrazione del contenuto di una pagina web indicata dall'utente, operazione affidata a un secondo servizio esterno dotato di una propria chiave: anche questa è custodita dal backend e non transita dal browser. Va dichiarato, per trasparenza verso l'utente e verso la proponente_G, **quali dati lasciano il perimetro del sistema_G**: verso il fornitore dei modelli linguistici transitano il testo sottoposto all'operazione — la porzione selezionata o l'intera nota — e le istruzioni costruite dall'applicazione; verso il servizio di estrazione transita il solo indirizzo fornito dall'utente. Nessuno dei due riceve dati identificativi dell'utente, che il prodotto non raccoglie.

Il confine è dato da ciò che l'utente sottopone, non dalla provenienza del testo: **nessun file è trasmesso automaticamente**. Una nota aperta da disco resta nel browser finché l'utente non ne affida il contenuto a un'operazione assistita; da quel momento esce dal perimetro come qualunque altro testo. Aprire un file non comporta dunque alcuna trasmissione, chiederne il riassunto sì: la differenza sta in un'azione che l'utente compie deliberatamente.

3. Rendering sicuro del Markdown_G. Il requisito_G **R-113-F-Ob** prevede una componente di rendering che trasformi il testo Markdown_G in una presentazione grafica formattata. Il testo restituito dal modello linguistico non è però contenuto fidato: è generato da un sistema_G esterno a partire da input_G che l'utente controlla solo in parte. Deve essere mostrato all'utente come HTML formattato senza che ciò apra la strada a vulnerabilità di tipo *cross-site scripting* (XSS). La

tecnologia di rendering deve quindi essere sicura per costruzione, non affidarsi a una sanificazione applicata a posteriori. Al medesimo criterio si riconduce **R-110-F-Ob**, che impone messaggi di errore in linguaggio naturale privi di dettagli tecnici: la gestione degli errori del modello non deve riversare nell'interfaccia contenuto proveniente dall'esterno.

4. Editor ricco ma stabile. L'editor deve offrire evidenziazione della sintassi, cronologia delle modifiche con annullamento e ripristino (**R-7-F-De**, **R-8-F-De**), e un comportamento accessibile da tastiera e da lettore di schermo; **R-111-F-Ob** ne prescrive la realizzazione come componente capace di ospitare testo su più righe. Deve inoltre mantenere invariati cursore e posizione di scorrimento mentre il pannello di anteprima si aggiorna in streaming, condizione che esclude soluzioni in cui la ricostruzione della vista coinvolge l'intera area di lavoro.

5. Manutenibilità_G con un gruppo di sette persone. Il prodotto è sviluppato in parallelo da sette persone con livelli di esperienza eterogenei. Ciò richiede codice tipizzato, in cui gli errori di interfaccia emergano in fase di compilazione anziché in esecuzione, confini netti fra i livelli dell'applicazione e verifica_G automatica delle modifiche prima della loro integrazione_G. Il criterio è vincolato da tre requisiti_G di qualità: **R-4-Q-Ob** richiede che ogni pull request_G sia sottoposta a revisione tra pari e superi i test_G automatici prima dell'integrazione_G nel ramo principale, **R-2-Q-Ob** richiede una copertura del codice_G conforme alle soglie del Piano di Qualifica, **R-10-Q-Ob** impone il rispetto delle Norme di Progetto_G. Serve quindi un'infrastruttura di *Continuous Integration_G* che applichi questi controlli in modo automatico e uniforme sui due livelli.

6. Perimetro tecnologico ammesso dal capitolato_G. Il requisito di vincolo_G **R-7-V-Op** stabilisce che la componente server-side, se realizzata, possa essere sviluppata utilizzando **Java** o **Python**. La scelta di Python ricade quindi all'interno del perimetro tecnologico ammesso dalla proponente_G. Sul versante client, **R-1-V-Ob** circoscrive il perimetro alle Tecnologie Web_G e ne fissa gli ambienti di esecuzione di riferimento: le versioni recenti di **Google Chrome**, **Mozilla Firefox** e **Microsoft Edge**. Questo vincolo_G ha conseguenza diretta sulla strategia di persistenza descritta nella sezione 2.4, poiché le tre famiglie di browser non offrono le medesime interfacce di accesso al file system.

Filosofia trasversale. Le scelte che seguono rispondono a un unico criterio di fondo: **l'equilibrio fra controllo e velocità di sviluppo_G**. Il gruppo ha preferito tecnologie che lasciano visibile e modificabile ciò che accade, evitando tanto le soluzioni che nascondono il comportamento dietro astrazioni ampie e opinate, quanto quelle che impongono di riscrivere da zero funzionalità già disponibili. A parità di adeguatezza tecnica, la preferenza è andata agli ecosistemi maturi, con documentazione estesa e ampia disponibilità di risorse: fattore determinante per un gruppo che affronta gran parte di queste tecnologie per la prima volta.

2.2 Frontend

Il frontend_G è un'applicazione a pagina singola eseguita interamente nel browser. Le tecnologie sono presentate nell'ordine con cui si sovrappongono: il linguaggio, la libreria che struttura la vista, l'ecosistema di librerie che ne realizza le funzioni specifiche e, infine, gli strumenti che ne governano la costruzione.

Tabella 1: Tecnologie adottate per il frontend

Tecnologia	Versione	Descrizione e ruolo nel progetto
TypeScript	6.0.3	Linguaggio di sviluppo _G dell'intero frontend _G . Estende JavaScript con un sistema di tipi statici che viene rimosso in fase di compilazione (<i>type erasure</i>): i tipi non esistono a tempo di esecuzione e non introducono alcun costo prestazionale. Il loro contributo è spostare in compilazione errori che si manifesterebbero altrimenti in produzione, in particolare sulle interfacce fra componenti. La validazione dei dati provenienti dall'esterno resta a carico del backend, poiché i tipi non sopravvivono alla compilazione e non possono verificare nulla a tempo di esecuzione.
React	19.2.7	Libreria per la costruzione dell'interfaccia, organizzata a componenti. Editor, anteprima, barra degli strumenti e pannelli delle funzioni assistite dall'LLM _G sono componenti React distinti. Il meccanismo di <i>reconciliation</i> confronta la rappresentazione dell'interfaccia prima e dopo un cambiamento di stato e applica al DOM le sole modifiche effettivamente necessarie: durante lo streaming, in cui lo stato cambia decine di volte al secondo, questo evita la ricostruzione dell'intera pagina. L'applicazione non impiega alcun router : l'interazione si svolge in una sola vista, priva di navigazione fra pagine distinte, e l'introduzione di un instradamento lato client sarebbe una complessità non giustificata.
Zustand	5.0.14	Gestione dello stato applicativo condiviso fra i componenti (testo della nota, selezione corrente, testo in arrivo dallo streaming, stato di errore, modalità di visualizzazione). Espone selettori granulari, ciascuno dei quali osserva una singola porzione di stato: durante lo streaming la ridipintura interessa il solo pannello di anteprima, mentre l'editor resta immobile e conserva cursore e posizione di scorrimento.
react-markdown	10.1.0	Rendering del Markdown _G nel pannello di anteprima. Il testo viene analizzato e trasformato in un albero sintattico astratto, dal quale la libreria costruisce direttamente gli elementi React corrispondenti. In nessun punto della catena viene iniettato HTML grezzo nel documento: la libreria è quindi sicura per costruzione rispetto agli attacchi XSS, senza necessità di un passaggio di sanificazione. Il requisito è critico, poiché il testo da rendere proviene in larga parte dal modello linguistico.
remark-gfm	4.0.1	Estensione della pipeline di analisi con le costruzioni <i>GitHub Flavored Markdown</i> (tabelle, elenchi di attività, testo barrato, collegamenti automatici), attese dall'utenza di riferimento del prodotto. Le tabelle in particolare non appartengono al Markdown _G di base: l'estensione è la condizione perché i requisiti _G R-34-F-De ... R-38-F-De (inserimento di una tabella e gestione di righe e colonne, UC44 ... UC48) trovino una resa nell'anteprima.

Tecnologia	Versione	Descrizione e ruolo nel progetto
remark-math rehype-katex KaTeX	6.0.0 7.0.1 0.17.0	Riconoscimento delle espressioni matematiche nel sorgente <code>Markdown_G</code> e loro composizione tipografica nell'anteprima. Realizzano i requisiti _G R-28-F-De e R-29-F-De (inserimento e rimozione di una formula scientifica o matematica, UC35 e UC37), che senza un motore di composizione dedicato non sarebbero verificabili _G sul lato dell'anteprima. Operano come estensioni della stessa pipeline ad albero sintattico, preservandone la proprietà di non iniettare HTML grezzo.
rehype-highlight highlight.js	7.0.2 11.11.1	Evidenziazione della sintassi all'interno dei blocchi di codice mostrati nell'anteprima. Discende dai requisiti _G R-26-F-De , R-27-F-De e R-119-F-De (inserimento, rimozione e modifica di un blocco di codice sorgente, UC32, UC34 e UC33), che presuppongono una resa del blocco di codice distinta dal testo ordinario.
Tailwind CSS	4.3.1	Sistema di stile <i>utility-first</i> : la presentazione è espressa mediante classi elementari applicate direttamente al marcatore, anziché tramite fogli di stile separati. Ne deriva un insieme chiuso di valori (spaziature, colori, dimensioni tipografiche) che mantiene l'aspetto uniforme fra le parti dell'interfaccia scritte da persone diverse. Il plugin <code>@tailwindcss/typography</code> (0.5.20) fornisce gli stili tipografici applicati al risultato del rendering <code>Markdown_G</code> .
shadcn/ui Radix UI	— 1.5.0	Insieme di componenti di interfaccia (pulsanti, aree di testo, finestre di dialogo, avvisi). <code>shadcn/ui</code> non è una dipendenza installata ma un catalogo da cui il codice sorgente dei componenti viene copiato nel progetto: resta quindi interamente leggibile e modificabile dal gruppo, senza vincolo di adozione verso un fornitore esterno. I componenti poggiano su Radix UI, che fornisce le primitive di comportamento accessibile — navigazione da tastiera, gestione del focus, attributi per i lettori di schermo — la cui realizzazione manuale corretta sarebbe onerosa e soggetta a errori.
CodeMirror 6	6.0.2	Editor di testo del pannello sinistro. Fornisce nativamente evidenziazione della sintassi <code>Markdown_G</code> (modulo <code>@codemirror/lang-markdown</code> 6.5.0), cronologia delle modifiche con annullamento e ripristino (<code>@codemirror/commands</code> 6.10.3) e supporto all'accessibilità, a fronte di un ingombro dell'ordine del centinaio di kilobyte. L'architettura modulare consente di includere le sole estensioni effettivamente utilizzate.
lucide-react	1.18.0	Insieme di icone vettoriali impiegate nella barra degli strumenti e nei comandi dell'interfaccia.
Vite	8.0.16	Strumento di sviluppo _G e costruzione del frontend _G . In sviluppo _G serve i moduli ES nativamente al browser, senza ricostruire l'intero pacchetto a ogni modifica, e aggiorna i moduli modificati mantenendo lo stato dell'applicazione; in produzione genera il pacchetto ottimizzato. Costituisce inoltre la base di configurazione riutilizzata dall'infrastruttura di test descritta nella sezione 2.5.

Alternative considerate

- **Angular, Vue e Svelte al posto di React**

Il confronto documentato nel verbale interno del 12 maggio 2026 ha contrapposto React e Svelte. Svelte adotta un approccio a compilatore: elimina il Virtual DOM spostando in fase di compilazione il lavoro che React svolge in esecuzione, con prestazioni superiori e minore ingombro del pacchetto prodotto. Il suo ecosistema è però più giovane, con minore disponibilità di librerie mature per le esigenze specifiche del prodotto e minore reperibilità di risorse di apprendimento, fattore rilevante per un gruppo che affronta la tecnologia per la prima volta. Vue è a sua volta maturo, ma la via più diffusa per il rendering del Markdown_G nel suo ecosistema consiste nel produrre HTML e iniettarlo nel documento, esattamente il rischio che il criterio 3 impone di evitare. Angular offre un quadro strutturale completo e fortemente prescrittivo, sovradimensionato per un'applicazione a vista singola e con una curva di apprendimento incompatibile con i tempi del progetto. La determinante è che le librerie richieste — rendering Markdown_G sicuro, componenti accessibili, integrazione_G dell'editor — individuano React come primo ambiente di destinazione.

- **Redux, Context API e Jotai al posto di Zustand**

Redux impone una quantità di codice di supporto (azioni, riduttori, selettori memorizzati) sproporzionata rispetto alla dimensione dello stato da gestire. La Context API, disponibile nativamente in React, non offre sottoscrizioni granulari: ogni variazione del valore del contesto provoca la ridipintura di tutti i componenti che vi accedono, comportamento penalizzante durante lo streaming, in cui lo stato cambia con frequenza elevata e ridipingere l'editor comporterebbe la perdita della posizione del cursore. Jotai adotta un modello ad atomi tecnicamente adeguato, ma richiede di scomporre lo stato in unità elementari fin dall'inizio: Zustand ottiene il medesimo risultato con un modello concettuale più semplice e una superficie di apprendimento minore.

- **marked e markdown-it al posto di react-markdown**

Entrambe le librerie convertono il Markdown_G in una stringa HTML, che dovrebbe poi essere inserita nel documento tramite un meccanismo di iniezione diretta. Ciò renderebbe obbligatorio un passaggio esplicito di sanificazione, la cui correttezza dipenderebbe dalla configurazione adottata e andrebbe verificata_G separatamente. react-markdown rende il problema inesistente per costruzione.

- **MUI e Chakra UI al posto di shadcn/ui**

Offrono un numero maggiore di componenti già pronti, ma impongono un linguaggio visivo marcato, la cui personalizzazione richiede di intervenire contro le impostazioni predefinite della libreria; inoltre il codice dei componenti resta all'interno della dipendenza, non modificabile. L'alternativa opposta, i CSS Modules, non aggiunge alcuna dipendenza ma non fornisce né un sistema di valori condiviso né componenti accessibili, che andrebbero realizzati integralmente dal gruppo.

- **Monaco e textarea nativa al posto di CodeMirror**

Monaco è l'editor impiegato in Visual Studio Code: è progettato per la scrittura di codice sorgente e la sua dotazione di funzioni — completamento automatico, analisi del linguaggio, riquadro di anteprima — eccede quanto richiesto dalla redazione di note, a fronte di un ingombro di oltre un ordine di grandezza superiore. L'elemento `textarea` nativo non comporta alcuna dipendenza, ma è privo di evidenziazione della sintassi e di una cronologia delle modifiche strutturata.

- **Webpack e Create React App al posto di Vite**

Ricostruiscono l'intero pacchetto a ogni modifica, con tempi di attesa in sviluppo_G sensibilmente superiori, e richiedono una configurazione più estesa e frammentata. Create React App non è

inoltre più oggetto di manutenzione attiva.

2.3 Backend

Il backend è una componente server che espone alcune API_G tramite HTTP. Le sue responsabilità sono quattro:

1. custodire le chiavi di accesso ai fornitori esterni, così che non raggiungano mai il browser (criterio 2);
2. comporre le istruzioni da inviare al modello linguistico;
3. ritrasmettere al browser il testo prodotto man mano che diviene disponibile (criterio 1);
4. acquisire il contenuto testuale di una pagina web indicata dall'utente, quando la generazione avviene a partire da un collegamento.

La quarta responsabilità discende dal requisito_G **R-74-F-De** (UC67.2 e UC67.5) e non è riducibile alle altre tre: comporta un secondo fornitore esterno, distinto da quello dei modelli, e per questo è trattata separatamente nel paragrafo *Estrazione del contenuto delle pagine web*. Come per il frontend_G, le tecnologie sono presentate nell'ordine linguaggio, framework_G, librerie, strumenti.

Tabella 2: Tecnologie adottate per il backend

Tecnologia	Versione	Descrizione e ruolo nel progetto
Python	3.12	Linguaggio di sviluppo _G del backend. Rientra nel perimetro tecnologico ammesso dal requisito _G R-7-V-Op. Dispone del supporto nativo alla programmazione asincrona (<code>async/await</code>), condizione necessaria per gestire attese di rete prolungate senza bloccare il processo, e dell'ecosistema più esteso per l'integrazione _G con i modelli linguistici.
FastAPI	0.136.1	Framework _G web su cui è costruita l'API _G . È asincrono per impostazione, requisito imprescindibile per lo streaming; deriva la validazione dei corpi delle richieste direttamente dalle annotazioni di tipo, senza codice di controllo scritto a mano; genera automaticamente lo schema OpenAPI dell'interfaccia esposta. La risposta in streaming è realizzata mediante StreamingResponse , che consuma un generatore asincrono e gestisce la trasmissione a blocchi lungo la connessione HTTP.
Pydantic	2.13.4	Validazione e serializzazione dei dati in ingresso e in uscita. Gli schemi Pydantic costituiscono la fonte unica di verità dei contratti dell'API _G : da essi FastAPI deriva lo schema OpenAPI e, da quest'ultimo, vengono generati i tipi TypeScript utilizzati dal frontend _G (si veda la sezione 2.6). Il contratto è definito quindi una volta sola, in un solo punto del sistema _G .
pydantic-settings	2.14.1	Lettura e validazione della configurazione applicativa (indirizzo del gateway dei modelli, identificativo del modello, chiave di accesso, origini ammesse dalla politica CORS) a partire dalle variabili d'ambiente. Le impostazioni sono esposte tramite un'unica istanza per processo.

Tecnologia	Versione	Descrizione e ruolo nel progetto
httpx	0.28.1	Client HTTP asincrono impiegato per la comunicazione verso il provider dei modelli linguistici. Supporta la lettura della risposta a blocchi man mano che arriva, senza attenderne il completamento, e propaga l'annullamento lungo l'intera catena: se l'utente interrompe la generazione, la chiusura si trasmette dal browser al backend e da questo alla connessione verso il provider, senza che rimangano richieste attive prive di destinatario.
tavily-python	0.7.26	Estrazione del contenuto testuale delle pagine web nella funzione di generazione a partire da un collegamento (requisito _G R-74-F-De , UC67.2 e UC67.5). Restituisce il solo testo della pagina, già ripulito dal marcatore e dagli elementi accessori, e ne consente il troncamento a una lunghezza massima prima dell'invio al modello. È un client verso un servizio esterno: si veda il paragrafo <i>Estrazione del contenuto delle pagine web</i> per il perimetro dei dati trasmessi e per l'alternativa scartata.
Uvicorn	0.47.0	Server ASGI sul quale l'applicazione FastAPI è posta in esecuzione. Costituisce l'implementazione dell'interfaccia asincrona fra il server HTTP e il codice applicativo.
Server-Sent Events	—	Protocollo di trasporto dei blocchi testuali dal backend al browser. È un formato testuale unidirezionale che viaggia su una normale connessione HTTP mantenuta aperta, in cui ogni evento è una riga con prefisso data: e la fine del flusso è segnalata da un marcatore convenzionale. La direzione unica — dal server al client — corrisponde esattamente alla natura del problema.

Accesso ai modelli linguistici. L'accesso ai modelli avviene attraverso un **gateway LiteLLM**, componente esterna che espone un'interfaccia compatibile con quella di OpenAI e uniforma l'accesso a fornitori diversi. Poiché si tratta di un servizio contattato via HTTP e non di una libreria installata nel progetto, non figura fra le dipendenze del backend e non è associato a una versione dichiarata nei file di progetto.

La comunicazione con il gateway è confinata dietro l'astrazione **LLMClient**, una classe astratta che dichiara la sola operazione di produzione incrementale del testo. L'implementazione concreta che dialoga con LiteLLM — interpretandone il formato di risposta e traducendone gli errori — ne è l'unico realizzatore, secondo il pattern Adapter descritto nella sezione dedicata all'architettura. **Né i router né la logica di dominio_G invocano direttamente LiteLLM:** conoscono unicamente l'interfaccia astratta. La sostituzione del gateway con un fornitore diverso comporta pertanto la scrittura di una nuova implementazione dell'interfaccia, e nessuna modifica al resto del backend.

Estrazione del contenuto delle pagine web. Il requisito_G **R-74-F-De** consente all'utente di indicare un URL come fonte di contesto per la generazione automatica di testo (UC67.2). Poiché il modello linguistico non accede alla rete, il contenuto della pagina deve essere acquisito dal prodotto e passato al modello come parte delle istruzioni. L'operazione è affidata a **Tavily**, servizio esterno che restituisce il testo di una pagina già ripulito dal marcatore, dalla navigazione e dagli elementi accessori.

Si tratta del **secondo fornitore esterno** del sistema_G, dotato di una propria chiave di accesso. Coerentemente con il criterio 2, la chiave è custodita dal backend e non raggiunge il browser; verso il servizio transita esclusivamente l'indirizzo indicato dall'utente, non il testo della

nota. Il contenuto restituito è troncato a una lunghezza massima prima di essere inoltrato al modello, così da mantenere prevedibile la dimensione delle istruzioni.

L'alternativa considerata è l'**estrazione lato server con una libreria locale: trafilatura** e le realizzazioni Python dell'algoritmo *Readability* scaricano la pagina con il client HTTP già presente nel progetto e ne isolano il contenuto principale senza coinvolgere alcun terzo. Eliminerebbero un fornitore, una chiave e un punto di uscita dei dati — vantaggio non trascurabile — ma sposterebbero sul gruppo la qualità dell'estrazione, che su pagine costruite dinamicamente tramite JavaScript risulta sensibilmente inferiore, e richiederebbero di gestire in proprio reindirizzamenti, codifiche e tempi di risposta dei siti contattati. Poiché R-74-F-De è desiderabile e non obbligatorio, il gruppo ha preferito la soluzione che ne rende prevedibile il completamento, accettando la dipendenza esterna.

Rispetto all'accesso ai modelli linguistici **non sussiste alcuna asimmetria**: le due dipendenze esterne sono trattate allo stesso modo. Come l'accesso al gateway passa dall'interfaccia astratta `LLMClient`, così l'estrazione passa dall'interfaccia astratta `ContentExtractor`, dichiarata anch'essa fra le porte del dominio_G e affiancata dal proprio tipo di errore, così che chi la consuma possa trattarne il fallimento senza conoscere l'implementazione che lo solleva. Ciascuna delle due porte ha un unico realizzatore concreto fra gli adattatori verso l'esterno, ciascuna è costruita da un provider dedicato nel medesimo modulo di composizione, e ciascuna raggiunge la rotta che se ne serve per iniezione della dipendenza: la rotta di generazione a partire da un collegamento dichiara entrambe le porte fra i propri parametri e non nomina alcun fornitore.

La conseguenza è la stessa rivendicata per il gateway: **sostituire il servizio di estrazione comporta la scrittura di un nuovo adattatore e la modifica del solo modulo di composizione**, non l'intervento sul dominio_G o sulle rotte. La differenza fra le due dipendenze non riguarda perciò la loro sostituibilità, ma il perimetro dei dati e la criticità: l'assenza della chiave del gateway impedisce l'avvio del processo, poiché senza modello linguistico nessuna funzione assistita è servibile, mentre l'assenza della chiave del servizio di estrazione degrada la sola generazione a partire da un collegamento e viene segnalata per singola richiesta.

Alternative considerate

• Node.js con TypeScript e Go al posto di Python

Entrambi sarebbero tecnicamente adeguati: Node.js consentirebbe di adottare un unico linguaggio sui due livelli, Go offrirebbe prestazioni superiori nella gestione di connessioni concorrenti prolungate. Nessuno dei due rientra però nel perimetro tecnologico ammesso dal requisito_G R-7-V-Op, che indica Java e Python. Java rientra nel perimetro ed è stato valutato, ma è stato scartato per la minore immediatezza del suo ecosistema nell'integrazione_G con i modelli linguistici e per la maggiore verbosità rispetto alla dimensione contenuta del backend, che espone poche interfacce e non contiene logica di dominio_G estesa.

• Flask e Django al posto di FastAPI

Flask è sincrono per impostazione: il supporto allo streaming asincrono richiederebbe estensioni aggiuntive e una configurazione non prevista dal suo modello di base, oltre a un livello di validazione scritto separatamente. Django è orientato ad applicazioni complete, con mappatore relazionale a oggetti, sistema di autenticazione e pannello di amministrazione integrati: componenti che il prodotto non utilizza, dato che non prevede persistenza su base di dati (si veda la sezione 2.4), e che ne renderebbero l'adozione sproporzionata.

• requests e aiohttp al posto di httpx

La libreria `requests` è lo standard di fatto per le richieste HTTP in Python, ma è esclusivamente sincrona: bloccherebbe il processo per l'intera durata della risposta del modello, vanificando la

scelta di un framework_G asincrono. La libreria `aiohttp` è asincrona e supporta lo streaming, ma espone un'ergonomia meno lineare nella gestione della chiusura anticipata delle connessioni, aspetto centrale per la funzione di annullamento della generazione.

- **SDK diretto del fornitore e LangChain al posto del gateway**

L'impiego dell'SDK proprietario di un singolo fornitore legherebbe il codice a quel fornitore, in un ambito in cui modelli e condizioni di accesso cambiano con frequenza elevata. LangChain, all'estremo opposto, introduce un livello di astrazione ampio e fortemente opinato — catene, agenti, memoria, strumenti — di cui il prodotto non utilizzerebbe che una frazione minima, a fronte di un aumento sensibile della superficie di dipendenza e di una minore prevedibilità del comportamento. Il gateway, unito all'interfaccia `LLMClient`, offre l'indipendenza dal fornitore senza alcuno dei due inconvenienti.

- **WebSocket e polling al posto dei Server-Sent Events**

I WebSocket stabiliscono un canale bidirezionale, con un protocollo di negoziazione dedicato e la necessità di gestire esplicitamente riconessioni e stato della connessione: capacità superflue, poiché nel prodotto i dati scorrono in una sola direzione. Il *polling* periodico richiederebbe di accumulare il testo prodotto lato server in attesa di essere richiesto, introducendo latenza e complessità di gestione dello stato intermedio.

2.4 Persistenza delle note

Il gruppo ha deliberato di non realizzare la persistenza delle note su base di dati. Le note non lasciano dunque la macchina dell'utente: sono salvate e ricaricate sul **file system locale** per il tramite del browser, e conservate fra una sessione e l'altra nell'**archivio locale del browser** stesso. Entrambi i meccanismi sono lato client; nessun dato dell'utente raggiunge il servizio applicativo per esservi memorizzato.

Meccanismo adottato. I meccanismi di persistenza lato client sono **due, distinti per scopo e non alternativi**. L'accesso al file system serve all'**esportazione e all'importazione deliberate**: è l'utente a chiedere esplicitamente di salvare una nota su disco o di aprirne una, e il file prodotto è un documento che gli appartiene e che può usare con altri strumenti. La persistenza nel browser serve invece alla **continuità della sessione**: senza di essa chiudere la scheda comporterebbe la perdita delle note in lavorazione, poiché non esiste alcun archivio server-side che le conservi. Il primo meccanismo è descritto di seguito, il secondo nel paragrafo *Continuità della sessione*.

Le operazioni di salvataggio e di caricamento su file sono realizzate con una strategia a due vie, selezionata a tempo di esecuzione in base alle capacità del browser ospite.

Quando il browser espone la **File System Access API** — verificata mediante la presenza dei metodi `showOpenFilePicker` e `showSaveFilePicker` sull'oggetto globale della finestra — l'applicazione apre la finestra di selezione file nativa del sistema_G operativo. Il caricamento ottiene un riferimento al file scelto e ne legge il contenuto testuale; il salvataggio ottiene un riferimento al percorso di destinazione e vi scrive un flusso di byte. L'annullamento da parte dell'utente è riconosciuto e trattato come esito legittimo, non come errore.

Quando l'interfaccia non è disponibile — condizione che ricorre su **Firefox** e **Safari**, che non la implementano — l'applicazione ricade su meccanismi supportati universalmente. La doppia via non è una precauzione generica: il requisito di vincolo_G **R-1-V-Ob** include Firefox fra gli ambienti di esecuzione di riferimento, quindi una realizzazione fondata sulla sola File System Access API lascerebbe scoperto uno dei tre browser prescritti. In caricamento crea un elemento `input` di tipo `file` e ne legge il contenuto tramite un lettore di file; in salvataggio costruisce un

oggetto **Blob** con il contenuto della nota, ne ricava un indirizzo temporaneo e lo assegna a un collegamento di scaricamento attivato per via programmatica, delegando al browser la scelta della destinazione secondo le impostazioni dell'utente. L'indirizzo temporaneo viene rilasciato al termine dell'operazione.

Il criterio di selezione è quindi la disponibilità effettiva dell'interfaccia nel browser in uso, non l'identificazione del browser stesso: l'applicazione non interroga l'agente utente, ma verifica direttamente la presenza dei metodi richiesti. La funzione resta pertanto disponibile su ogni browser, con la sola differenza che sui browser dotati dell'interfaccia nativa l'utente sceglie esplicitamente il percorso di destinazione, mentre sugli altri il file segue le impostazioni di scaricamento predefinite.

Continuità della sessione. Il secondo meccanismo non richiede alcuna azione dell'utente. Lo store delle note è avvolto in un middleware di persistenza che ne riversa lo stato nel **localStorage** del browser sotto la chiave **notes_persistence**, e lo rilegge all'avvio dell'applicazione: alla riapertura l'elenco delle note e la nota corrente sono quelli con cui la sessione precedente si era chiusa, e il contenuto della nota corrente è ricaricato nell'editor. Ciò che viene conservato è l'elenco delle note con i rispettivi identificativo, titolo, contenuto e istanti di creazione e di ultima modifica, oltre all'indicazione di quale sia la nota corrente.

Va precisato **quando** il testo in lavorazione raggiunge quell'archivio, per non attribuire al meccanismo una copertura che non ha: il contenuto dell'editor è riversato nella nota corrente in occasione del cambio di nota e della creazione di una nuova nota, non a ogni modifica del testo né alla chiusura della scheda. I metadati, assegnati alla creazione, sono invece persistiti immediatamente. Le modifiche successive all'ultimo cambio di nota non sono quindi conservate: è un limite noto della realizzazione attuale, non una proprietà del supporto.

I limiti del supporto sono quelli propri dell'archiviazione locale del browser e vanno dichiarati. Lo spazio disponibile è soggetto a una **quota**, il cui superamento fa fallire la scrittura: l'applicazione lo tratta come condizione prevista e ripristina lo stato precedente anziché lasciare l'interfaccia in uno stato incoerente. I dati sono **confinati all'origine e al profilo del browser** in uso: non sono raggiungibili da un altro browser, da un altro dispositivo o da una finestra di navigazione anonima, non vi è alcuna sincronizzazione fra dispositivi, e la cancellazione dei dati di navigazione li rimuove. Questo meccanismo **non è un sostituto della persistenza server-side**: resta interamente lato client, e non copre alcuno dei requisiti_G opzionali elencati più avanti in questa sezione.

Formato dei file. Le note sono salvate in **Markdown_G**, con estensione **.md** e tipo MIME **text/markdown**; in caricamento sono accettate anche le estensioni **.txt**. La scelta del testo semplice attua il requisito di vincolo_G **R-4-V-Ob**, che prescrive la memorizzazione delle note salvate localmente come file di testo. Il file contiene il solo testo della nota, senza intestazioni o marcatori aggiuntivi: resta quindi leggibile e modificabile con qualsiasi altro strumento, e non introduce alcun vincolo_G verso il prodotto.

Metadati della nota. I metadati che l'applicazione associa alla nota — identificativo, titolo, istante di creazione e istante di ultima modifica — non sono scritti nel file, che resta un documento **Markdown_G** puro. Il titolo è veicolato dal nome del file: in salvataggio il nome proposto deriva dal titolo della nota, in caricamento il titolo è ricavato dal nome del file privato dell'estensione. La derivazione avviene in **entrambi** i rami di caricamento, quello nativo e quello di ripiego, con un titolo convenzionale riservato al solo caso in cui il file scelto non esponga un nome utilizzabile: la parità fra i due rami è necessaria, poiché quello di ripiego è il percorso primario su Firefox, incluso fra i browser prescritti dal requisito_G di vincolo_G **R-1-V-Ob**.

Sul requisito_G **R-97-F-De** (UC84), che richiede la visualizzazione delle informazioni e dei metadati di una nota selezionata, vanno distinti **due casi**, che non hanno la medesima copertura.

- **Nota creata nel prodotto.** Identificativo, titolo e istanti di creazione e di ultima modifica sono assegnati alla creazione e affidati alla persistenza locale descritta sopra, che li scrive immediatamente: **sopravvivono alla chiusura della sessione** e sono disponibili alla riapertura, alle condizioni proprie di quel supporto (stessa origine, stesso profilo del browser). Per queste note il requisito_G è coperto dai dati che l'applicazione già possiede, senza che nulla debba essere ricavato dal file system.
- **Nota importata da disco.** Il file contiene il solo testo, quindi i metadati vanno ricostruiti da ciò che il browser espone sull'oggetto **File**. Il titolo lo è già, come appena descritto; gli istanti no: all'importazione entrambi sono posti all'istante di apertura, che descrive quando la nota è entrata nell'applicazione e non quando il file è stato scritto o modificato.

Trattandosi di un requisito_G **desiderabile** e non opzionale, il secondo caso non può essere rinviato.

Per le note importate il recupero è ottenuto senza modificare il formato del file, sfruttando i metadati che il file system già espone. L'oggetto **File** restituito sia dalla File System Access API sia dall'elemento **input** di tipo file riporta gli attributi **name**, **lastModified** e **size**: all'apertura di una nota l'applicazione li impiega per popolare rispettivamente titolo, istante di ultima modifica e dimensione del contenuto. Il numero di caratteri e il numero di parole sono derivati dal testo caricato. Resta non ricostruibile il solo **istante di creazione**, che il file system espone in modo non uniforme fra le piattaforme e che le interfacce del browser non rendono disponibile: per le sole note importate da disco tale informazione è presentata come non disponibile anziché con un valore convenzionale, così che l'interfaccia non affermi un dato che il prodotto non possiede. Per le note create nel prodotto l'istante di creazione è invece un dato posseduto e conservato, e va presentato come tale.

Motivazione dell'esclusione della base di dati. La persistenza server-side è prevista dal capitolato_G come estensione facoltativa. I requisiti_G funzionali che ne derivano sono classificati come opzionali nell'Analisi dei Requisiti_G e riguardano quattro capacità distinte:

- **operazioni sull'archivio remoto:** R-83-F-Op (creazione di una nota nella base di dati, UC74.2), R-86-F-Op (caricamento, UC76.2), R-89-F-Op (salvataggio, UC78.2), R-92-F-Op (rimozione, UC80.2), R-95-F-Op (rinomina, UC82.2). Sono il nucleo di ciò che l'esclusione comporta: le quattro operazioni di base sulle note, più la rinomina;
- **ricerca e filtro:** R-98-F-Op (ricerca per titolo), R-99-F-Op (ricerca full-text), R-100-F-Op e R-101-F-Op (filtro per data di creazione e di ultima modifica);
- **autenticazione:** R-102-F-Op (accesso con credenziali), R-103-F-Op (gestione degli errori di autenticazione), R-108-F-Op (disconnessione);
- **gestione dell'account:** R-104-F-Op (registrazione), R-105-F-Op (unicità dello username), R-106-F-Op (conferma della password), R-107-F-Op (rimozione dell'account e dei dati associati).

Sono opzionali anche i requisiti_G di vincolo_G corrispondenti: **R-5-V-Op** (memorizzazione delle note in una base di dati raggiungibile dal sistema_G), **R-6-V-Op** (componente server raggiungibile tramite chiamate HTTP) e **R-7-V-Op** (linguaggio della componente server-side). Si segnala che dei tre soltanto R-7-V-Op è già soddisfatto dall'architettura attuale, poiché la componente

server esiste per le funzioni assistite da LLM_G indipendentemente dalla persistenza; R-5-V-Op e R-6-V-Op restano scoperti insieme alla funzionalità che li condiziona.

Il gruppo ha scelto di concentrare le risorse della presente iterazione sul completamento dei requisiti_G obbligatori e desiderabili, in particolare sulle funzionalità assistite da LLM_G , che costituiscono il nucleo del prodotto richiesto dalla proponente_G. Lo stato di copertura di ciascun requisito_G è riportato nella sezione 5.

Va precisato l'effettivo margine di estensione, per non attribuire all'architettura una predisposizione che non possiede. Nel perimetro attuale la persistenza delle note risiede **interamente nel frontend_G**: le note non raggiungono in alcun momento il backend, e nel dominio_G del backend non esiste una porta per le note in attesa di essere implementata. Introdurre la persistenza su base di dati comporterebbe quindi lavoro nuovo e non marginale: entità di dominio_G per la nota e per l'utente, le porte corrispondenti, gli endpoint che le espongono, un adattatore verso il sistema_G di gestione della base di dati e l'intero blocco di autenticazione richiesto da R-102-F-Op ... R-108-F-Op.

Ciò che l'architettura esagonale garantisce è un'altra proprietà, più circoscritta ma sufficiente: la persistenza sarebbe aggiunta **come nuova porta e nuovo adattatore**, senza perturbare il dominio_G delle funzioni assistite da LLM_G già realizzato. Il costo dell'estensione ricade sulle componenti nuove, non sulla riscrittura di quelle esistenti.

2.5 Tecnologie di test e analisi

Le attività di verifica_G previste dalle Norme di Progetto_G si articolano in analisi statica_G, condotta sul codice sorgente senza porlo in esecuzione, e analisi dinamica_G, condotta eseguendo il codice su casi di prova predisposti.

2.5.1 Analisi statica

L'analisi statica_G intercetta gli errori prima che il codice venga eseguito, spostandone il rilevamento dalla fase di prova a quella di scrittura.

Tabella 3: Strumenti di analisi statica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Compilatore TypeScript	6.0.3	Verifica _G della coerenza dei tipi sull'intero codice del frontend _G . La compilazione è eseguita in modo esplicito come primo passo del comando di costruzione (<code>tsc -b && vite build</code>): un errore di tipo interrompe la costruzione e impedisce la produzione del pacchetto. La configurazione attiva inoltre la segnalazione di variabili e parametri dichiarati e non utilizzati, dei casi non terminati nei costrutti di selezione multipla e delle costruzioni non rimovibili per sola cancellazione dei tipi.
ESLint	10.5.0	Analisi statica _G del codice del frontend _G rispetto a un insieme di regole configurate. Individua costrutti sintatticamente validi ma problematici, che il compilatore non segnala.
typescript-eslint	8.61.0	Estensione di ESLint alla sintassi TypeScript e alle regole specifiche del linguaggio.

Tecnologia	Versione	Descrizione e ruolo nel progetto
eslint-plugin-react-hooks	7.1.1	Verifica _G del rispetto delle regole di impiego degli <i>hook</i> di React, in particolare della completezza degli elenchi di dipendenze. Si tratta di una classe di errori che non produce alcun messaggio in esecuzione ma si manifesta come stato dell'interfaccia non aggiornato, di difficile diagnosi.
eslint-plugin-react-refresh	0.5.2	Verifica _G che i moduli rispettino le condizioni necessarie all'aggiornamento a caldo dei componenti durante lo sviluppo _G .
Ruff	0.16.1	Analisi statica _G e formattazione automatica del codice del backend. Riunisce in un solo strumento le regole di più analizzatori dell'ecosistema Python: gli insiemi selezionati sono quelli di pyflakes , pycodestyle (errori e avvisi), isort , pyupgrade e flake8-bugbear , che segnalano importazioni e variabili non utilizzate, ordinamento delle importazioni, costrutti sintatticamente validi ma problematici e violazioni delle convenzioni di stile. Il formattatore integrato riproduce lo stile di black , indicato dalle Norme di Progetto _G per la formattazione del codice Python: la conformità è quindi mantenuta senza introdurre un secondo strumento. La configurazione risiede in ruff.toml , accanto ai file di progetto del backend.
mypy	—	Verifica _G statica delle annotazioni di tipo del backend, in modalità rigorosa. Controlla la coerenza fra le firme dichiarate e le chiamate effettive, la completezza delle annotazioni e la correttezza dei tipi dei generatori asincroni impiegati nello streaming — classe di errori che, in un linguaggio a tipizzazione dinamica, si manifesterebbe altrimenti solo in esecuzione e, nel caso dello streaming, a operazione già avviata. Porta il backend al medesimo livello di controllo che il compilatore TypeScript garantisce sul frontend _G .

L'analisi statica_G è quindi applicata a **entrambi** i livelli del prodotto, come richiesto dalle Norme di Progetto_G e, per il loro tramite, dal requisito_G di qualità **R-10-Q-Ob**. La simmetria fra i due livelli è voluta: al compilatore TypeScript ed ESLint sul frontend_G corrispondono mypy e Ruff sul backend, con la medesima ripartizione fra verifica_G dei tipi e verifica_G delle regole di scrittura.

Si osserva che la validazione operata dagli schemi **Pydantic** non appartiene a questa categoria e non ne costituisce un sostituto: Pydantic controlla i *dati* che attraversano i confini dell'applicazione a tempo di esecuzione, mentre l'analisi statica_G controlla il *codice* senza eseguirlo. Le due verifiche_G intercettano difetti disgiunti — un corpo di richiesta malformato nel primo caso, un'incoerenza fra firme di funzione nel secondo — e sono pertanto complementari.

2.5.2 Analisi dinamica

L'analisi dinamica_G è organizzata su test_G di unità e di integrazione_G dei due livelli, eseguiti separatamente dalle rispettive catene di strumenti.

Tabella 4: Strumenti di analisi dinamica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Vitest	4.1.8	Esecutore dei test _G di unità e di integrazione _G del frontend _G . Riutilizza integralmente la configurazione di Vite — risoluzione dei moduli, alias dei percorsi, trasformazione di TypeScript e JSX — evitando la manutenzione di una seconda catena di strumenti allineata alla prima.
@vitest/coverage-v8	4.1.8	Misurazione della copertura del codice _G raggiunta dai test _G del frontend _G . Si appoggia allo strumento di copertura nativo del motore V8, senza richiedere una strumentazione del codice sorgente. Le soglie minime sono dichiarate nella configurazione di Vitest, così che il superamento sia verificato _G dall'esecutore stesso e non affidato alla lettura del rapporto; l'insieme dei file misurati vi è dichiarato esplicitamente, così che un sorgente privo di test pesi sul denominatore anziché sparire dal calcolo. È la controparte di pytest-cov sul versante frontend _G : senza di essa il Piano di Qualifica non potrebbe riportare una metrica _G di copertura su questo livello, come richiesto da R-2-Q-Ob .
jsdom	29.1.1	Realizzazione del DOM in ambiente Node.js, che consente di eseguire i test _G dei componenti senza avviare un browser.
Testing Library (React)	16.3.2	Interrogazione dei componenti resi attraverso ruoli, etichette e testo visibile, così come vi accederebbe una persona che utilizza il prodotto, anziché attraverso la struttura interna del marcatore. I test _G restano quindi validi a fronte di riorganizzazioni interne dei componenti e verificano indirettamente l'accessibilità dell'interfaccia.
jest-dom user-event	6.9.1 14.6.1	Asserzioni specifiche per gli elementi del DOM e simulazione delle interazioni dell'utente (digitazione, selezione, attivazione di comandi) con la sequenza di eventi effettivamente prodotta da un browser.
pytest	9.0.3	Esecutore dei test _G del backend. Il sistema delle <i>fixture</i> consente di comporre e riutilizzare le precondizioni dei test _G in modo esplicito.
pytest-asyncio	1.4.0	Esecuzione dei test _G su funzioni asincrone, condizione necessaria data la natura asincrona dell'intero backend. La modalità automatica configurata nel progetto rende superflua l'annotazione di ogni singolo test _G .
pytest-cov	7.1.0	Misurazione della copertura del codice _G raggiunta dai test _G del backend.
respx	0.23.1	Interposizione sulle richieste HTTP effettuate tramite httpx. Consente di verificare _G il comportamento del client verso il provider dei modelli — flusso corretto, risposta malformata, errore di connessione, scadenza del tempo massimo — senza contattare alcun servizio esterno, rendendo i test _G deterministici e indipendenti dalla rete.

Alternative considerate

- **Jest al posto di Vitest**

Jest è lo standard storico dei test_G in ambiente JavaScript, con la comunità più ampia e la

maggiore disponibilità di documentazione. Richiederebbe però una configurazione separata e mantenuta in parallelo a quella di Vite, in particolare per la trasformazione di TypeScript e per i moduli ES, che Jest non supporta nativamente. Il disallineamento fra le due configurazioni è una fonte di errori ricorrente, e non è compensato da alcun vantaggio funzionale: l'interfaccia di programmazione di Vitest è deliberatamente compatibile con quella di Jest, e i `testG` risultano pressoché identici.

- **unittest al posto di pytest**

Il modulo `unittest` è incluso nella libreria standard di Python e non aggiunge dipendenze. Impone però una struttura a classi che rende i `testG` più verbosi e, soprattutto, non dispone né di un sistema di *fixture* componibili paragonabile a quello di `pytest` né dell'ecosistema di estensioni sul quale il progetto si appoggia per l'esecuzione asincrona, la misurazione della copertura del codice_G e l'interposizione sulle richieste HTTP.

2.6 Infrastruttura

Le tecnologie di questa sezione non concorrono al comportamento del prodotto in esecuzione, ma governano il modo in cui esso viene costruito, avviato, verificato_G e mantenuto coerente fra i due livelli.

Organizzazione della repository_G. Prima delle tecnologie va richiamata una decisione organizzativa che le condiziona tutte: il gruppo ha adottato un'unica repository_G (*mono-repo*) per i due livelli del prodotto. Il codice del frontend_G risiede in `/web` e quello del backend in `/api`, ciascuno con il proprio gestore di pacchetti, il proprio file di dipendenze e il proprio file di lock. Restano quindi due progetti distinti, con costruzioni e dipendenze proprie: condividono la repository_G e la cronologia delle modifiche, non divengono un unico artefatto_G.

La scelta ha due conseguenze dirette sulle tecnologie descritte di seguito. La prima è che una variazione del contratto dell'API_G e il corrispondente adeguamento del client possono essere integrati_G con una sola pull request_G, che li sottopone alla medesima verifica_G automatica: non esiste un intervallo in cui i due livelli risultino disallineati. La seconda è che la configurazione di verifica_G è definita una volta sola, in un solo insieme di flussi di lavoro.

Tabella 5: Tecnologie di infrastruttura

Tecnologia	Versione	Descrizione e ruolo nel progetto
Docker Docker Compose	29.4.3 5.1.4	Ambiente di sviluppo _G ed esecuzione riproducibile. Il file <code>docker-compose.yml</code> descrive i due servizi <code>web</code> e <code>api</code> , la rete che li collega, le porte esposte verso la macchina ospite e le variabili d'ambiente, e li avvia con un solo comando. Le immagini di base sono fissate dai rispettivi <code>Dockerfile</code> : <code>node:22-alpine</code> per il frontend _G e <code>python:3.12-slim</code> per il backend. La linea di Node.js non è scelta soltanto come limite inferiore: è fissata al di sopra e al di sotto dal campo <code>engines</code> di <code>package.json</code> (<code>>=22 <23</code>) e dal file <code>.nvmrc</code> , che l'ambiente locale e l'integrazione _G continua leggono entrambi. Il limite superiore ha una ragione misurata e documentata nel repository _G : su linee successive parte della suite di test _G del frontend _G non è eseguibile.

Tecnologia	Versione	Descrizione e ruolo nel progetto
GitHub Actions	—	Infrastruttura di <i>Continuous Integration</i> _G . Su ogni pull request _G , e su ogni integrazione _G nei rami principali, esegue in processi indipendenti l'analisi statica _G dei due livelli, la verifica _G dei contratti di dipendenza del backend, il controllo che il contratto dell'API _G e i tipi da esso derivati siano allineati al codice, le due suite di test _G con misurazione della copertura del codice _G e la costruzione del pacchetto di produzione del frontend _G . È il luogo in cui gli strumenti delle sezioni 2.5.1 e 2.5.2 vengono effettivamente applicati: senza di esso la loro esecuzione dipenderebbe dalla diligenza individuale. Trattandosi di un servizio della piattaforma e non di una dipendenza installata, non è associato a una versione dichiarata nei file di progetto; le versioni fissate sono quelle delle singole azioni richiamate dai flussi di lavoro.
pnpm	11.18.0	Gestore dei pacchetti del frontend _G . Memorizza ogni versione di ogni pacchetto una sola volta e la collega per riferimento, riducendo occupazione di spazio e tempi di installazione. Il file <code>pnpm-lock.yaml</code> fissa le versioni risolte dell'intero albero delle dipendenze; la versione dello strumento stesso è vincolata dal campo <code>packageManager</code> di <code>package.json</code> , che l'integrazione _G continua legge per approvvigionarsi, così che questa e lo sviluppo _G locale adoperino il medesimo risolutore. Il vincolo _G non si estende all'immagine di sviluppo _G del frontend _G , che installa lo strumento senza indicarne la versione: chi lavora nel container può quindi risolvere una versione diversa da quella dichiarata.
uv	0.11.16	Gestore dei pacchetti e dell'ambiente virtuale del backend. Risolve le dipendenze dichiarate in <code>pyproject.toml</code> e ne fissa le versioni in <code>uv.lock</code> ; l'installazione in fase di costruzione dell'immagine avviene in modalità vincolata al file di lock, così che l'ambiente sia identico su ogni macchina.
openapi-typescript	7.13.0	Generazione dei tipi TypeScript del client a partire dallo schema OpenAPI prodotto da FastAPI. Il frontend _G non dichiara autonomamente la forma dei dati scambiati con il backend: la deriva dal contratto pubblicato dal backend stesso. Una modifica di uno schema Pydantic si propaga così ai tipi del client e, se incompatibile con l'uso che il frontend _G ne fa, si manifesta come errore di compilazione anziché come guasto in esecuzione. Il file generato è escluso dall'analisi di ESLint, non essendo codice scritto a mano.

Nota sulle versioni degli strumenti di sviluppo. Le versioni indicate per **Docker**, **Docker Compose** e **uv** vanno lette diversamente da tutte le altre del documento. Quelle delle librerie sono fissate dai file di lock e risultano identiche su ogni macchina; queste tre no, perché nessun file del progetto le vincola: Docker e Compose sono presupposti dall'ambiente anziché dichiarati, e uv è installato dall'immagine del backend senza indicazione di versione e risolto alla versione corrente dall'integrazione_G continua. I valori riportati sono quindi quelli **in uso sull'ambiente di sviluppo del gruppo** alla data del documento, e non un vincolo_G verificabile_G: su un'altra macchina, o in un altro momento, l'installazione può risolvere versioni diverse. Per Docker il numero è quello di CLI e motore riportato da `docker --version`, che non coincide con la versione di Docker Desktop, la quale segue una numerazione propria. Fa eccezione **pnpm**, che il repository_G vincola: il valore riportato è quello dichiarato in `package.json` e non quello rilevato dall'ambiente.

Verifica automatica delle pull request_G. I controlli eseguiti dall'integrazione_G continua sono la condizione che ogni modifica deve soddisfare prima di essere integrata_G nel ramo principale. Sono organizzati in **tre processi indipendenti**, ciascuno con i propri passi:

- **frontend_G:** installazione vincolata al file di lock, ESLint, rigenerazione dei tipi dallo schema OpenAPI e confronto con il file versionato, compilazione dei tipi con **tsc**, esecuzione di Vitest con misurazione della copertura del codice_G e verifica_G delle soglie, costruzione del pacchetto di produzione con Vite;
- **backend:** installazione vincolata a **uv.lock**, Ruff nella modalità di controllo delle regole, riesportazione dello schema OpenAPI e confronto con il file versionato, esecuzione di pytest con **pytest-cov** e verifica_G della soglia;
- **contratti di dipendenza:** installazione vincolata a **uv.lock** e controllo dei contratti dichiarati per l'architettura del backend, descritti nella sezione 3.4.1.

La soglia di copertura verificata dai primi due processi è quella fissata dal Piano di Qualifica per il valore accettabile della metrica_G corrispondente; entrambi conservano il rapporto prodotto come artefatto_G del processo, così che il dato sia consultabile a posteriori e non solo nel registro di esecuzione.

L'esito negativo di un qualunque passo impedisce l'integrazione_G. Questi controlli soddisfano il requisito_G **R-4-Q-Ob**, che subordina l'integrazione_G nel ramo principale al superamento dei test_G automatici oltre che alla revisione tra pari, e rendono **verificabile** quanto la voce `openapi-typescript` afferma: perché un'incompatibilità di contratto si manifesti “come errore di compilazione” occorre che una compilazione avvenga a ogni modifica, in un ambiente che non si possa aggirare. Il controllo sul contratto è **doppio e simmetrico**: il processo del backend riesporta lo schema dal codice e pretende che coincida con `openapi.json` versionato, quello del frontend_G rigenera i tipi da quello schema e pretende che coincidano con il file versionato. Se il primo confronto fallisce, il contratto pubblicato non corrisponde più al codice che lo produce; se fallisce il secondo, lo schema è cambiato senza che il client sia stato aggiornato. In entrambi i casi la verifica_G si interrompe prima che il disallineamento raggiunga il ramo principale.

Nota sui container. Un container non è una macchina virtuale, bensì un normale processo del sistema_G operativo ospite, isolato mediante funzionalità del kernel: i *namespaces* gli attribuiscono una vista propria di processi, rete e file system, i *cgroups* ne limitano il consumo di risorse. Poiché il kernel è condiviso con la macchina ospite, anziché replicato come in una macchina virtuale, l'ingombro è sensibilmente inferiore e l'avvio richiede secondi anziché minuti.

Alternative considerate

• Configurazione manuale dell'ambiente al posto di Docker

Richiederebbe a ciascun membro del gruppo di installare e allineare manualmente le versioni di Node.js, di Python e delle rispettive dipendenze di sistema_G su tre famiglie di sistemi_G operativi differenti. Le configurazioni divergerebbero inevitabilmente, con la conseguenza che un comportamento osservato su una macchina non sarebbe riproducibile sulle altre — circostanza che rende la diagnosi dei guasti sproporzionatamente onerosa e vanifica il valore probatorio dei test_G.

• Podman al posto di Docker

Podman è compatibile con i formati e i comandi di Docker e presenta un vantaggio di sicurezza_G

non trascurabile: non richiede un processo demone privilegiato in esecuzione permanente e consente di eseguire i container senza privilegi amministrativi. È stato scartato per ragioni di diffusione, ampiezza della documentazione e familiarità pregressa dei membri del gruppo, elementi che riducono il tempo speso in attività non produttive.

- **Poly-repo al posto del mono-repo**

La separazione di frontend_G e backend in repository_G distinte è appropriata quando i due componenti sono sviluppati da gruppi diversi e rilasciati in modo indipendente. Non è il caso del progetto: le due parti sono sviluppate dallo stesso gruppo e rilasciate congiuntamente. La separazione imporrebbe due pull request_G coordinate a ogni variazione del contratto dell'API $_G$, con una finestra temporale in cui le due repository_G risultano disallineate, oltre alla duplicazione della configurazione di verifica_G .

- **Tipi TypeScript scritti a mano al posto della generazione da OpenAPI**

È l'alternativa che non aggiunge alcuno strumento, ma introduce una duplicazione del contratto dell'API $_G$ in due punti del sistema $_G$ mantenuti separatamente. Le due definizioni divergono inevitabilmente non appena il backend evolve senza che il frontend_G venga aggiornato di conseguenza; poiché i tipi TypeScript sono rimossi in compilazione, la divergenza non produce alcun errore e si manifesta come guasto silenzioso in esecuzione. La generazione a partire dallo schema elimina la duplicazione alla radice.

3 Architettura

3.1 Panoramica architetturale

La presente sezione fornisce la visione d'insieme del sistema_G: le unità che lo compongono, i confini che le separano e lo stile architetturale adottato all'interno di ciascuna. Ne fissa inoltre il **vocabolario**, che le sezioni successive impiegano senza introdurre sinonimi. Il dettaglio di ciascuna parte è rimandato a quelle sezioni: qui si stabilisce soltanto che cosa esiste, come è delimitato e secondo quale criterio è organizzato al proprio interno.

3.1.1 Visione d'insieme

Il sistema_G è composto da **due unità di deployment indipendenti**:

- un'**applicazione web a pagina singola** (*Single Page Application*), eseguita interamente nel browser dell'utente;
- un **servizio applicativo** che espone un'API_G HTTP.

Le due unità risiedono in altrettante cartelle della repository_G — **web/** e **api/** — ciascuna con il proprio gestore di pacchetti, le proprie dipendenze e la propria catena di costruzione: restano quindi due progetti distinti, costruiti ed eseguiti separatamente, e non due parti di un unico artefatto_G. Il modo in cui vengono poste in esecuzione è trattato nella sezione dedicata all'architettura di deployment.

La comunicazione fra le due unità avviene su **HTTP** secondo uno stile **REST**, ed è asimmetrica: l'applicazione web è l'unico soggetto che avvia le richieste, il servizio applicativo il solo che risponde. Le risposte delle operazioni assistite dal modello linguistico sono restituite **in streaming**: il testo raggiunge il browser a frammenti, man mano che il modello lo produce, e compare progressivamente nell'interfaccia anziché tutto insieme al termine dell'elaborazione. Il servizio non trattiene il testo per consegnarlo completo, ma lo inoltra appena disponibile; il formato con cui i frammenti viaggiano sulla connessione appartiene alla sezione sul flusso dei dati.

Il servizio applicativo è a sua volta **client di due servizi esterni**, entrambi contattati via HTTP e nessuno dei due realizzato dal gruppo:

- un **gateway LiteLLM**, che espone un'interfaccia compatibile con quella di OpenAI e per il cui tramite avviene l'inferenza del modello linguistico;
- **Tavily**, che restituisce il contenuto testuale di una pagina web il cui indirizzo è indicato dall'utente.

Il perimetro del sistema_G comprende quindi due unità realizzate dal gruppo e due dipendenze esterne raggiunte dalla sola unità server: l'applicazione web non contatta in alcun caso i due fornitori.

Non esiste persistenza lato server. Il servizio applicativo non possiede dati: non dispone di una base di dati, non conserva stato applicativo fra una richiesta e la successiva e il suo dominio_G non dichiara alcuna porta per le note o per gli utenti. I dati dell'utente restano nel browser. Il meccanismo con cui vi permangono è descritto altrove; qui rileva la sola conseguenza architetturale: il confine del sistema_G non racchiude alcun archivio, e le due unità non condividono alcuno stato persistente.

3.1.2 Stile architetturale del servizio applicativo

Il servizio applicativo adotta l'**architettura esagonale** (*Ports & Adapters*). Il criterio che la caratterizza è la direzione delle dipendenze: al centro stanno le regole del prodotto, attorno gli adattatori che le collegano al mondo esterno, e la dipendenza punta sempre verso il centro, mai in senso contrario.

Il nucleo. Il package `app/core/` contiene le regole del prodotto — i casi d'uso, i valori del dominio_G e le porte — e **non importa nulla dagli altri package**. Non conosce il framework_G web, non conosce il client HTTP, non conosce i fornitori esterni né il modulo che legge la configurazione. Un caso d'uso è una funzione che riceve i dati dell'operazione e, come ultimo parametro, la porta di cui si serve: non sa chi la implementi e non ha modo di scoprirlo.

Le porte. Le porte dichiarate sono **due**, entrambe in `app/core/ports/`. `LLMClient` dichiara la produzione incrementale di testo da parte di un modello linguistico; `ContentExtractor` dichiara l'estrazione del contenuto testuale di un indirizzo web. Sono interfacce astratte: descrivono ciò di cui il dominio_G ha bisogno, non il modo in cui un fornitore lo realizza. Accanto a ciascuna è dichiarato il tipo di errore che essa può sollevare, così che chi la consuma possa trattarne il fallimento senza conoscere l'implementazione che lo produce.

I due lati dell'esagono. Gli adattatori si dispongono su due lati opposti. In `app/api/` vive ciò che traduce una richiesta esterna in una chiamata al dominio_G: è il lato da cui l'applicazione viene invocata. In `app/infrastructure/` vive ciò che il dominio_G chiama per parlare con il mondo: è il lato che l'applicazione invoca. **Le due direzioni non si toccano direttamente:** un adattatore del primo lato non istanzia un adattatore del secondo, ma dichiara la porta di cui ha bisogno e la riceve dall'esterno. Il modulo che stabilisce quale adattatore soddisfa quale porta è uno solo — il *composition root* — ed è l'unico punto dell'applicazione in cui i due adattatori concreti sono nominati fuori dai moduli che li definiscono. Sostituire un fornitore esterno significa perciò scrivere un nuovo adattatore e cambiare una riga in quel modulo, non intervenire sul dominio_G.

Il vincolo_G non è affidato alla disciplina dei redattori. È il punto qualificante di questa architettura, e va detto esplicitamente: la direzione delle dipendenze **non è una convenzione che il gruppo si impegna a rispettare**, ma un vincolo_G dichiarato e verificato_G automaticamente. I contratti sono scritti in `api/.importlinter` e controllati staticamente dallo strumento `import-linter`, che ispeziona il grafo delle importazioni senza eseguire il codice. Sono quattro e stabiliscono che il nucleo non importi il framework_G web, il client HTTP, il client del servizio di estrazione, la libreria di configurazione né i package degli adattatori; che i livelli siano ordinati dall'esterno verso il centro; che gli adattatori non si importino fra loro; che il nucleo non importi il modulo di configurazione. Il controllo è un passo dell'integrazione_G continua, non un'abitudine locale: un'importazione che invertisse la dipendenza non sopravviverebbe fino alla revisione tra pari, perché farebbe fallire la verifica_G automatica prima.

Il beneficio ottenuto. Ciò che l'esagono restituisce, e che è verificabile_G nel codice, è la **provabilità del dominio_G in isolamento**: un caso d'uso si esercita senza montare HTTP e senza rete, passandogli al posto della porta un doppio che ne rispetta il contratto. I test_G dei casi d'uso del prodotto sono scritti esattamente così, e non istanziano alcun adattatore. La proprietà non è un corollario teorico dello stile adottato: discende dal fatto che il tipo del parametro sia

la porta e mai l'adattatore, condizione che i contratti di dipendenza rendono impossibile violare inavvertitamente.

Lo stile qui dichiarato — un servizio applicativo unico, organizzato a esagono — è stato adottato in alternativa alla scomposizione in microservizi e al monolite a strati tradizionale. L'argomentazione del confronto, con i costi e i benefici di ciascuna alternativa, è riportata nelle sezioni 3.2.2 e 3.2.3.

3.1.3 Stile architetturale dell'applicazione web

L'applicazione web è organizzata **a componenti, con stato condiviso esterno all'albero e flusso dei dati unidirezionale**.

I componenti React compongono l'interfaccia per annidamento: ciascuno riceve dal genitore i dati che deve presentare e ne restituisce la rappresentazione. Lo stato realmente condiviso non risiede però nell'albero dei componenti: vive fuori di esso, in **due store distinti**, che i componenti interrogano direttamente senza doverlo trasmettere lungo la gerarchia. Il flusso resta unidirezionale: un'interazione dell'utente invoca un'azione dello store, l'azione aggiorna lo stato, i componenti sottoscritti alla porzione modificata si ridisegnano. Non esiste un percorso inverso in cui un componente modifichi la vista di un altro.

La regola seguita nel collocare lo stato è graduale: **stato locale al componente per difetto**, sollevato al genitore comune quando due componenti vicini devono concordare, **promosso a store soltanto quando componenti lontani nell'albero devono condividerlo**. Lo store non è quindi il deposito di tutto lo stato dell'applicazione: le condizioni puramente locali di un componente — l'apertura di un pannello, il testo in corso di digitazione in un campo, la voce dell'elenco in fase di rinomina — restano nel componente che le possiede, e non se ne ha traccia altrove.

I componenti si sottoscrivono a **single porzioni** di stato, non allo stato intero: ciascuno store espone un selettore per ogni porzione osservabile, e il componente che ne dichiara uno viene ridisegnato solo quando quella porzione cambia. È la proprietà che rende sostenibile lo streaming: mentre il testo arriva a frammenti e la porzione che lo raccoglie cambia molte volte al secondo, si ridisegna la sola parte di interfaccia che presenta l'esito in arrivo, mentre l'editor — sottoscritto a una porzione distinta, il testo della nota — non viene toccato. Una sottoscrizione allo stato nel suo complesso, o a un oggetto che ne aggrega più porzioni, riporterebbe il ridisegno sull'intera area di lavoro a ogni frammento ricevuto.

I due store si dividono le responsabilità in questo modo: `useEditorStore` custodisce lo stato della sessione di scrittura, cioè il testo corrente, la selezione, l'esito in arrivo dal servizio applicativo, la condizione di errore e di avanzamento e la modalità di visualizzazione; `useNotesStore` custodisce la raccolta delle note e l'indicazione di quale sia quella corrente. La loro composizione interna appartiene alla sezione sull'architettura logica dell'applicazione web.

3.1.4 Vocabolario

I termini fissati nella tabella 6 sono impiegati con questo significato in tutto il capitolo. Le sezioni successive vi si attengono e non ne adottano sinonimi.

Tabella 6: Vocabolario architetturale

Termine	Definizione
Porta	Interfaccia astratta dichiarata dal nucleo, che descrive ciò di cui il dominio _G ha bisogno dall'esterno senza indicare come venga realizzato.

Termine	Definizione
Adattatore primario	Modulo che traduce una richiesta proveniente dall'esterno in una chiamata al dominio _G ; risiede in <code>app/api/</code> .
Adattatore secondario	Modulo che realizza una porta traducendo la chiamata del dominio _G nel protocollo di un servizio esterno; risiede in <code>app/infrastructure/</code> .
Nucleo (dominio_G)	Insieme delle regole del prodotto, contenuto in <code>app/core/</code> , che non importa nulla dagli altri package del servizio applicativo.
Caso d'uso	Funzione del nucleo che realizza una singola operazione del prodotto, ricevendo i dati dell'operazione e, come ultimo parametro, la porta di cui si serve.
Composition root	Unico modulo che stabilisce quale adattatore soddisfa quale porta, e unico punto in cui gli adattatori concreti sono nominati fuori dai moduli che li definiscono.
Store	Contenitore di stato condiviso dell'applicazione web, esterno all'albero dei componenti, che espone lo stato in lettura e le azioni che lo modificano.
Selettore	Funzione che estrae da uno store una singola porzione di stato e alla quale un componente si sottoscrive; determina quando quel componente viene ridisegnato.

3.2 Architettura di deployment

3.2.1 Unità di esecuzione e configurazione

3.2.2 Confronto con l'architettura a microservizi

3.2.3 Confronto con il monolite a strati

3.3 Architettura del frontend

3.3.1 Organizzazione dei sorgenti

3.3.2 Scomposizione in componenti

3.3.3 Gestione dello stato applicativo

3.3.4 Integrazione dell'editor

3.3.5 Accesso al servizio applicativo

3.3.6 Persistenza lato client

3.4 Architettura del backend

La presente sezione descrive l'architettura logica del servizio applicativo: i package in cui il codice è ripartito, il criterio che vi assegna un modulo, le responsabilità di ciascuna parte e le firme con cui esse collaborano. Lo stile adottato, l'architettura esagonale, è dichiarato nella sezione 3.1.2: qui se ne documenta la realizzazione. Il perimetro si arresta al confine HTTP: quanto sta oltre è nella sezione 3.3, la messa in esecuzione nella sezione 3.2, i design pattern nella sezione 3.5.

Notazione degli elenchi di firme. Il segno + contrassegna i membri pubblici, il segno - quelli che il modulo non espone all'esterno. Le funzioni di modulo, che in Python non appartengono ad alcuna classe, sono riportate senza segno. I tipi sono quelli annotati nel codice.

Diagramma delle classi del backend: le porte `LLMClient` e `ContentExtractor` con la rispettiva operazione e il rispettivo tipo di errore; i realizzatori `LiteLLMClient` e `TavilyExtractor` con le dipendenze verso i due fornitori esterni; i sette casi d'uso di `core/services/` e le loro dipendenze verso le porte; il value object `Message` e i tipi di `core/domain/values.py`; gli otto router di `api/routes/`, i sette DTO di `api/schemas.py` e lo schema `ErrorResponse`; l'adattatore di trasporto `sse_response`; il composition root `dependencies.py` con i tre provider. Le frecce di dipendenza sono tutte dirette verso il centro.

Figura 1: Architettura del servizio applicativo

3.4.1 Struttura a esagono e contratti di dipendenza

Il codice del servizio applicativo risiede in `api/app/` ed è ripartito in tre package e alcuni moduli di radice. Il criterio che assegna un modulo all'uno o all'altro non è la somiglianza tematica ma la **direzione della dipendenza**: un modulo appartiene al nucleo se non ha bisogno di conoscere nulla del mondo esterno, appartiene a un lato dell'esagono se conosce un protocollo, un framework_G o un fornitore.

- `app/core/`: il nucleo. Contiene i casi d'uso (`core/services/`), i valori e le istruzioni del dominio_G (`core/domain/`) e le porte (`core/ports/`). Non importa nulla dagli altri package e nulla dalle librerie che realizzano il trasporto.
- `app/api/`: gli adattatori primari. Contiene le rotte HTTP (`api/routes/`), gli schemi di validazione delle richieste (`api/schemas.py`) e la traduzione degli errori di validazione in messaggi rivolti all'utente (`api/errors.py`). È il lato da cui l'applicazione viene invocata.
- `app/infrastructure/adapters/`: gli adattatori secondari. Contiene i due moduli che realizzano le porte parlando con i rispettivi fornitori e il modulo che dà forma alla risposta in streaming. È il lato che l'applicazione invoca.
- I moduli di radice, che descrivono non una responsabilità del prodotto ma il modo in cui il processo è messo insieme: `main.py` costruisce l'applicazione e vi monta i router, `dependencies.py` è il composition root, `settings.py` dichiara lo schema della configurazione, `export_openapi.py` scrive su file il contratto dell'API_G.

Nessun package prende il nome di una tecnologia: la regola di denominazione è enunciata nella sezione 3.6.3.

Verifica dei contratti di dipendenza. La direzione delle dipendenze è dichiarata in `api/.importlinter` come **quattro contratti**, verificati da `import-linter`, che ispeziona il grafo delle importazioni senza eseguire il codice. Il controllo è uno dei tre processi dell'integrazione_G continua descritti nella sezione 2.6: un'importazione che invertisse la dipendenza farebbe fallire la verifica_G automatica prima di arrivare alla revisione tra pari.

Tabella 7: Contratti di dipendenza dichiarati in `api/.importlinter`

Contratto	Tipo	Che cosa stabilisce
core-indipendente	<i>forbidden</i>	Il nucleo non importa il framework _G web (<code>fastapi</code>), il client HTTP (<code>httpx</code>), il client del servizio di estrazione (<code>tavily</code>), la libreria di configurazione (<code>pydantic_settings</code>) né i due package degli adattatori (<code>app.api</code> , <code>app.infrastructure</code>).
layers	<i>layers</i>	I tre package sono ordinati dall'esterno verso il centro: <code>app.api</code> sopra <code>app.infrastructure</code> , <code>app.infrastructure</code> sopra <code>app.core</code> . Un livello può importare quelli sottostanti, mai quelli soprastanti.
adapter-indipendente	<i>independence</i>	I tre moduli di <code>app/infrastructure/adapters/</code> (<code>litellm_client</code> , <code>tavily_extractor</code> , <code>sse_streaming</code>) non si importano fra loro.
core-non-importa-settings	<i>forbidden</i>	Il nucleo non importa <code>app.settings</code> : non conosce nemmeno lo schema della configurazione del processo.

Portata del contratto layers. L'ordinamento in livelli consente a un adattatore primario di importare da `app/infrastructure/`, ed è ciò che accade: ognuna delle sette rotte assistite importa `sse_response`, che è un modulo di trasporto interno. Nessuna rotta **istanzia però un adattatore di fornitore**: i nomi `LiteLLMClient` e `TavilyExtractor` non vi compaiono, perché la rotta dichiara la porta e la riceve dal composition root (sezione 3.4.5).

3.4.2 Adattatori primari

Il package `app/api/` traduce una richiesta HTTP in una chiamata al nucleo. Espone **otto rotte**, ciascuna definita in un modulo proprio di `api/routes/` e montata su `main.py` come router indipendente: sette accettano un corpo in POST e realizzano altrettante funzioni assistite dal modello linguistico (`/api/summarize`, `/api/generate`, `/api/generate-from-link`, `/api/translate`, `/api/rewrite`, `/api/grammar`, `/api/critique`); l'ottava, `GET /api/constants`, non accetta corpo e pubblica nel contratto le soglie del dominio_G tramite lo schema `ApiConstants`, dichiarato nel modulo della rotta stessa. A queste si aggiunge la rotta di salute `GET /`, che risiede in `main.py` perché non appartiene ad alcuna funzione del prodotto.

Forma comune delle sette rotte assistite. Ciascuna si esaurisce in tre passi e non contiene logica di dominio_G. Il primo non è scritto in alcuna riga: quando l'handler viene eseguito il corpo è già stato validato, perché la validazione è dichiarata dal tipo del parametro. Il secondo invoca il caso d'uso, che costruisce le istruzioni e interroga la porta. Il terzo consegna il flusso all'adattatore di trasporto. La porta arriva alla rotta per iniezione della dipendenza, come valore predefinito del parametro corrispondente: **nessuna rotta nomina un adattatore concreto** e nessuna importa le istruzioni destinate al modello.

Il caso d'uso è importato con l'alias `<nome>_service` perché **il nome dell'handler non è libero**: FastAPI ne deriva l'`operationId` pubblicato in `openapi.json`, dal quale il generatore di tipi del frontend_G ricava i nomi esposti al client. L'alias evita la collisione con il nome della funzione del nucleo senza toccare il contratto.

Schemi di validazione. Il modulo `api/schemas.py` contiene i sette DTO delle richieste (`TextRequest`, `GenerateRequest`, `LinkRequest`, `TranslateRequest`, `RewriteRequest`,

`GrammarRequest`, `CritiqueRequest`) e lo schema `ErrorResponse`, unica forma di ogni risposta di errore dell'API_G. I vincoli_G di lunghezza e i vocabolari ammessi sono importati da `core/domain/values.py`: il confine dichiara *che* un campo è vincolato_G, non *quanto*, così che la soglia non esista in due copie destinate a divergere. Accanto ai DTO sta `FIELD_LABELS`, che associa a ciascun campo la propria etichetta in italiano, perché aggiungere un campo e la sua etichetta sia la stessa modifica; un campo assente dalla mappa produce un messaggio generico anziché tecnico.

Traduzione degli errori di validazione. Il modulo `api/errors.py` traduce un errore di validazione di Pydantic in una frase di linguaggio naturale, come impone il requisito_G **R-110-F-Ob**. La forma predefinita di FastAPI espone i campi tecnici `type`, `loc` e `ctx` e rimanda al mittente il campo `input`, cioè il testo che l'utente ha scritto: **nessuno dei quattro raggiunge la risposta**. La traduzione copre le classi di errore che l'applicazione può produrre — campo mancante, testo troppo corto o troppo lungo, valore fuori dal vocabolario ammesso, indirizzo malformato, corpo non in formato JSON — e per ciascuna compone una frase con causa e azione correttiva. Davanti a un valore fuori vocabolario **non elenca i valori ammessi**: quell'elenco è una struttura interna di Pydantic e arriva in lingua inglese, mentre l'interfaccia propone già all'utente le sole opzioni valide.

Forma unica delle risposte di errore. Il modulo `main.py` registra due gestori che riconducono ogni esito negativo allo schema `ErrorResponse`: uno per le eccezioni HTTP sollevate dalle rotte, uno per gli errori di validazione, restituiti con stato 422. Alle sette rotte con corpo è associata la dichiarazione esplicita che la risposta 422 ha forma `ErrorResponse`: senza di essa il contratto pubblicherebbe lo schema generato in automatico da FastAPI, che descrive `detail` come sequenza di oggetti, forma che l'applicazione non produce.

L'adattatore di trasporto. La funzione `sse_response` converte il flusso di frammenti prodotto dal caso d'uso in una risposta HTTP progressiva. Risiede in `app/infrastructure/adapters/` e non in `app/api/` perché conosce il formato degli eventi sul filo, che è un protocollo: il criterio che assegna un modulo a un package è la conoscenza che esso possiede, non il luogo da cui viene chiamato. Il formato degli eventi è nella sezione 4.1.

Alla firma appartiene una scelta che va motivata qui: la funzione **estrae il primo frammento prima di restituire la risposta**. Il server invia l'intestazione di stato prima di iniziare a iterare il flusso, e dopo quel momento non è più possibile rispondere con uno stato di errore; il prelievo anticipato intercetta quindi un guasto del fornitore mentre lo stato è ancora modificabile e lo traduce in 503 (UC72). Un guasto che sopraggiunga a metà flusso richiede un trattamento diverso (sezione 4.5). La funzione verifica inoltre a ogni frammento se il client si è disconnesso e in tal caso chiude il flusso: è il tratto terminale della propagazione dell'annullamento trattata nella sezione 3.6.1.

Firme.

- `summarize(payload: TextRequest, request: Request, client: LLMClient): StreamingResponse`: e con la medesima forma `generate`, `translate`, `rewrite`, `grammar` e `critique`, ciascuna con il proprio DTO. Il parametro `client` è dichiarato con `Depends(get_llm_client)`.
- `generate_from_link(payload: LinkRequest, request: Request, client: LLMClient, extractor: ContentExtractor): StreamingResponse`: unica rotta a

dichiarare due porte fra i propri parametri; il flusso che ne deriva è descritto nella sezione 4.3.

- `constants(): ApiConstants`: pubblica nel contratto il sentinella della correzione grammaticale e le quattro soglie di lunghezza, tipizzate come letterali così che compaiano nello schema come costanti e non come stringhe o interi generici.
- `describe_validation_error(error: dict): str`: traduce un singolo errore di validazione nella frase rivolta all'utente.
- `sse_response(request: Request, stream: AsyncGenerator[str, None], log_label: str, logger: Logger | None): StreamingResponse`: adattatore di trasporto.

3.4.3 Il nucleo

Il package `app/core/` contiene ciò che il prodotto fa, separato da ogni conoscenza di come venga invocato e di chi esegua per suo conto le operazioni verso l'esterno. Si articola in tre parti: i **casi d'uso** in `core/services/`, il **dominio_G** in `core/domain/` e le **porte** in `core/ports/`.

Forma comune dei sette casi d'uso. La forma è stata fissata dal primo dei sette, il riassunto (UC62), e i sei successivi vi si attengono.

- **Una funzione, non una classe.** L'unica dipendenza del caso d'uso è la porta, e la porta è un parametro. Una classe con un costruttore e un solo metodo di esecuzione offrirebbe la stessa iniezione e la stessa provabilità in più righe.
- **Un modulo per caso d'uso, chiamato come l'operazione.** Il modulo `summarize.py` espone la funzione `summarize`.
- **La porta è l'ultimo parametro**, dopo i dati dell'operazione: riassumi *questo testo*, di *questa lunghezza*, tramite *questa porta*.
- **Il tipo del parametro è la porta, mai l'adattatore.** Il caso d'uso non sa se dall'altra parte vi sia il gateway dei modelli o un doppio predisposto per un test_G. Da questa regola discende la provabilità del dominio_G in isolamento dichiarata nella sezione 3.1.2, e il contratto `layers` ne impedisce la violazione.
- **Il caso d'uso restituisce il flusso della porta senza consumarlo.** Chi lo invoca riceve un flusso che non ha ancora prodotto nulla e non ha contattato nessuno. Ne discendono due comportamenti visibili all'utente: un guasto all'apertura resta traducibile in uno stato HTTP (sezione 3.4.2) e il testo compare progressivamente. Ne discende anche che **l'eccezione del fornitore non viene sollevata quando il caso d'uso è chiamato**, ma alla prima iterazione del flusso restituito.

L'unica eccezione alla forma. La generazione a partire da un collegamento è il solo caso d'uso a dipendere da **due** porte e il solo dichiarato `async def`: gli altri sei sono funzioni sincrone. Nessuno dei sette è scritto come generatore con `yield`, perché il corpo di un generatore non viene eseguito finché qualcuno non ne consuma il flusso; qui la differenza conta, dato che l'estrazione del contenuto della pagina deve avvenire all'atto della chiamata per restare traducibile in uno stato HTTP (sezione 4.3).

I valori del dominio_G. Il modulo `core/domain/values.py` raccoglie ciò che le funzioni assistite condividono: i vocabolari ammessi (le tre lunghezze, le quattro lingue di destinazione della traduzione, i tre registri della riscrittura, i sei cappelli dell'analisi critica), le soglie di lunghezza minima e massima del testo e delle istruzioni, e il sentinella con cui la correzione grammaticale segnala di non aver trovato errori. Sono dichiarati come tipi letterali: il valore ammesso e il tipo che lo descrive sono la stessa dichiarazione, quindi non è possibile cambiarne uno senza l'altro.

Il modulo contiene inoltre il value object `Message`, immutabile, con i due campi `role` e `content`: è il tipo che porte e istruzioni si scambiano. Il modello del dominio_G si ferma qui, senza un'entità per la conversazione intera, senza un tipo enumerato per il ruolo e senza involucri attorno ai frammenti in uscita, dove la stringa è già il tipo giusto.

La composizione delle istruzioni destinate al modello. Il package `core/domain/prompts/` costruisce le istruzioni ed è dominio_G: che cosa il modello possa dire e in che forma debba scriverlo è una decisione di prodotto, non una proprietà del fornitore. Quattro moduli con quattro compiti distinti:

- `rules.py` dichiara ogni regola **una volta sola**, come costante nominata, e ne consente il riuso fra istruzioni diverse.
- `composer.py` mette in fila le parti — il ruolo attribuito al modello, le regole di contenuto, le regole di forma e, dove l'utente può sceglierla, la lunghezza richiesta — e ne ricava i due messaggi da inviare. **Non è il pattern Builder**: la sezione 3.5.5 argomenta perché.
- `templates.py` contiene i **sette costruttori**, uno per funzione assistita, dai quali risultano dodici istruzioni distinte: l'analisi critica ne produce una per ciascuno dei sei cappelli, le altre sei operazioni una ciascuna. La distinzione fra le dodici è verificata_G da un test_G dedicato. Nel modulo resta ciò che è proprio della singola operazione; **una regola che comparisse in due costruttori appartiene a `rules.py`**.
- `untrusted.py` contiene la sola trasformazione che agisce sul contenuto di terze parti.

Confinamento del contenuto non fidato. Per ogni operazione vale la regola che **il testo dell'utente finisce esclusivamente nel messaggio a lui destinato**, mai fra le istruzioni di prodotto; i test_G la verificano_G su tutti e sette i costruttori. Alla sola generazione a partire da un collegamento si aggiunge il confinamento del materiale estratto: il contenuto viene racchiuso fra due marcatori, e i marcatori che comparissero *dentro* di esso vengono resi inerti, altrimenti una pagina potrebbe chiudere il recinto per conto proprio e il testo successivo si presenterebbe come istruzione. La sostituzione avviene carattere per carattere e non allunga il testo, perché il troncamento alla lunghezza massima avviene *prima* della composizione.

Il **confine della mitigazione è più stretto di quanto sembri, e va dichiarato**: il confronto sui marcatori è esatto, quindi una variante scritta in minuscolo o spezzata da spazi attraversa il contenuto intatta e, pur non aprendo nulla, un modello linguistico potrebbe leggerla come una chiusura. Anche la regola che dichiara non autorevole il contenuto estratto **dice al modello di ignorare le istruzioni che vi comparissero, non gli impedisce di obbedirvi**.

Le due porte. Sono dichiarate in `core/ports/` come classi astratte, ciascuna con una sola operazione e con il tipo di errore che essa può sollevare. L'errore sta con la porta perché **fa parte del contratto**: chi la consuma deve poterlo intercettare senza sapere quale implementazione lo produca, e se visse nell'adattatore il contratto `layers` impedirebbe all'adattatore di trasporto di importarlo. Nessuna delle due porte dichiara invece un'operazione di chiusura,

benché entrambi i realizzatori possiedano risorse da rilasciare: il ciclo di vita di quelle risorse è un fatto dell'adattatore, e dichiararlo nella porta obbligherebbe ogni doppio predisposto per un `testG` a implementare un metodo che non ha nulla da chiudere.

Firme.

- `+stream(messages: Sequence[Message]): AsyncIterator[str]`: unica operazione della porta `LLMClient`. Solleva `LLMProviderError`. Il parametro è una sequenza di `Message` e non di dizionari: la forma di filo del fornitore sta oltre la porta. È dichiarato come sequenza e non come lista perché la porta legge e non modifica.
- `+extract(url: str): str`: unica operazione della porta `ContentExtractor`. Solleva `ContentExtractorError`. La porta **non promette alcun limite di lunghezza** sul risultato: il troncamento è politica del realizzatore, e dichiararlo qui significherebbe vincolare_G a rispettarla anche i doppi dei `testG`.
- `summarize(text: str, length: Length, llm: LLMClient): AsyncIterator[str]`
- `generate(prompt: str, length: Length, llm: LLMClient): AsyncIterator[str]`
- `translate(text: str, target_language: Language, llm: LLMClient): AsyncIterator[str]`
- `rewrite(text: str, style: Style, llm: LLMClient): AsyncIterator[str]`
- `grammar(text: str, llm: LLMClient): AsyncIterator[str]`: unico dei sette a non impiegare alcun vocabolario del dominio_G oltre al testo, perché la correzione non ha parametri da scegliere.
- `critique(text: str, hat: Hat, llm: LLMClient): AsyncIterator[str]`
- `generate_from_link(url: str, length: Length, extractor: ContentExtractor, llm: LLMClient): AsyncIterator[str]`: le due porte sono gli ultimi parametri, nell'ordine in cui vengono impiegate. Solleva `InvalidLinkError` e `FetchError`, dove il primo tipo è sottotipo del secondo: i due fallimenti hanno esiti HTTP distinti (collegamento scartato senza toccare la rete, estrazione fallita), ma chi non volesse distinguerli può intercettare il solo tipo generale.
- `validate_link(url: str): None`: scarta il collegamento inutilizzabile prima di qualunque richiesta di rete.
- `fetch_and_extract(url: str, extractor: ContentExtractor): str`: interroga la porta di estrazione e riconduce il suo errore al tipo del caso d'uso, troncando il risultato alla lunghezza massima ammessa dal dominio_G.
- `compose(role: str, user: str, content_rules: Sequence[str], form_rules: Sequence[str], content_heading: str, length: Length | None): list[Message]`: solleva `ValueError` se il ruolo o il testo dell'utente sono vuoti, che sono i due elementi senza i quali l'istruzione non è una richiesta.
- `wrap_extracted_content(content: str): str`: neutralizza i marcatori e racchiude fra essi il contenuto estratto.

3.4.4 Adattatori secondari

Il package `app/infrastructure/adapters/` contiene ciò che il nucleo invoca per parlare con il mondo. Ciascuna delle due porte ha **un solo realizzatore concreto**, responsabile di due traduzioni: dalla chiamata del dominio_G al protocollo del proprio fornitore, e dagli errori del

fornitore al tipo di errore che la porta dichiara. Sono le due traduzioni che consentono al nucleo di ignorare l'esistenza dei fornitori. Entrambi i moduli realizzano il pattern definito nella sezione 3.5.1; la provenienza delle chiavi di accesso è nella sezione 3.2.1.

LiteLLMClient, realizzatore di LLMClient. Possiede un client HTTP asincrono configurato con l'indirizzo del gateway, un tempo massimo di attesa e l'intestazione di autorizzazione. L'operazione di flusso apre una richiesta di completamento chiedendo la risposta progressiva, la legge riga per riga man mano che arriva e restituisce i frammenti di testo che ne estrae. È l'**unico punto del backend in cui un Message assume la forma di filo del fornitore**: le chiavi con cui ruolo e contenuto viaggiano sulla rete sono protocollo, e la traduzione occupa una riga sola.

La conversione degli errori riconduce a `LLMProviderError` quattro condizioni distinte: gateway irraggiungibile, tempo massimo scaduto, risposta con stato di errore e risposta malformata. **Nessuna eccezione del client HTTP attraversa il confine dell'adattatore**, perché chi consuma la porta dovrebbe altrimenti conoscere la libreria di trasporto per trattarla.

TavilyExtractor, realizzatore di ContentExtractor. Restituisce il contenuto testuale di una pagina, già ripulito dal marcatore e dagli elementi accessori, troncato a una lunghezza massima. Il troncamento è **politica dell'adattatore e non del contratto**: la porta non promette alcun limite, perché nessun altro realizzatore sarebbe tenuto a rispettarlo. Ogni fallimento — rete, permessi, pagina inesistente, pagina priva di testo — viene ricondotto a `ContentExtractorError`.

Una proprietà dell'adattatore non è visibile nella firma e va documentata: **il client del fornitore compie l'operazione di rete in modo sincrono**, e invocarlo direttamente da una funzione asincrona bloccherebbe il ciclo di eventi per l'intera durata della richiesta, fermando ogni altra richiesta in corso **compresi i flussi già aperti**. L'adattatore lo esegue perciò su un thread separato.

Ciclo di vita e configurazione. Entrambi gli adattatori possiedono un pool di connessioni che vive quanto il processo ed espongono un'operazione di chiusura asincrona, invocata alla terminazione. La simmetria delle due firme è voluta anche dove non sarebbe necessaria: la chiusura di `TavilyExtractor` non compie operazioni di rete e potrebbe essere sincrona, ma è dichiarata asincrona perché chi la invoca non debba distinguere i due casi. Nessuno dei due, inoltre, legge la configurazione del processo: la riceve dal costruttore, perché leggerla all'interno legherebbe una classe di infrastruttura alla configurazione globale e renderebbe impossibile istanziarla in un `test_G` senza alterare l'ambiente di esecuzione.

Indipendenza reciproca degli adattatori. Il contratto **adapter-indipendente** vieta a ciascuno dei tre moduli del package di importare gli altri due. L'elenco comprende anche l'adattatore di trasporto descritto nella sezione 3.4.2: pur non realizzando alcuna porta, risiede in questo package perché conosce un protocollo, e soggiace quindi allo stesso vincolo_G.

Firme.

- `+LiteLLMClient(settings: Settings)`: costruisce il client HTTP con indirizzo, tempo massimo e intestazione di autorizzazione ricavati dalla configurazione ricevuta.
- `+stream(messages: Sequence[Message]): AsyncIterator[str]`: realizzazione dell'operazione dichiarata dalla porta `LLMClient`.

- `-_parse_sse_line(line: str): str | None`: operazione statica; estrae il frammento di testo da una riga della risposta del gateway e restituisce il valore nullo per le righe da ignorare. Solleva `LLMProviderError` sulle righe malformate.
- `+aclose(): None`: chiude il client HTTP sottostante.
- `+TavilyExtractor(api_key: str)`: solleva `ContentExtractorError` se la chiave ricevuta è vuota.
- `+extract(url: str): str`: realizzazione dell'operazione dichiarata dalla porta `ContentExtractor`.
- `+aclose(): None`: smonta il pool di connessioni del client del fornitore.

3.4.5 Composition root e configurazione

Il modulo `app/dependencies.py` è il composition root del servizio applicativo: **l'unico modulo che nomina `LiteLLMClient` e `TavilyExtractor` fuori dai moduli che li definiscono**. Da qui discende quanto affermato nella sezione 3.1.2: sostituire un fornitore esterno comporta la scrittura di un nuovo adattatore e la modifica di questo solo modulo. Il meccanismo con cui le porte raggiungono le rotte è il pattern definito nella sezione 3.5.4.

Non esistono provider dei casi d'uso. I casi d'uso sono funzioni la cui unica dipendenza è un parametro: non c'è nulla da costruire, perché la rotta si fa iniettare la porta e la passa alla funzione. Un provider per ciascun caso d'uso sarebbe un livello di indirizzione che non inietterebbe nulla che la rotta non abbia già ricevuto.

Tre provider, una istanza per processo. I provider sono la configurazione, l'adattatore del modello linguistico e l'adattatore di estrazione, ciascuno memoizzato. Poiché nessuno riceve argomenti, la memoizzazione ha una chiave sola: il corpo viene eseguito alla prima chiamata e ogni chiamata successiva restituisce l'oggetto già costruito. Ne risulta **una istanza per processo**, esito voluto per due componenti che possiedono ciascuna un pool di connessioni. Che questa costruzione **non** sia il Singleton della classificazione di Gamma e altri è argomentato nella sezione 3.5.5. Il provider della configurazione, a differenza degli altri due, non è iniettato nelle rotte: a invocarlo sono la costruzione dell'applicazione e i provider stessi, e i `test_G` lo governano alterando le variabili d'ambiente e azzerandone la memoizzazione.

Le due chiusure interrogano la memoizzazione. Alla terminazione vanno rilasciate le risorse dei soli adattatori effettivamente costruiti: le due funzioni di chiusura verificano perciò se la memoizzazione contenga già un'istanza e, in caso contrario, non fanno nulla. Non è un risparmio di lavoro. Entrambi i provider rifiutano di costruire l'adattatore quando la chiave corrispondente manca, quindi invocarli in chiusura farebbe fallire l'uscita del processo in ogni ambiente privo di configurazione, a cominciare da quello dei `test_G`.

Le due funzioni risiedono qui e non fra le operazioni di avvio e terminazione (sezione 3.2.1) perché la strategia di memoizzazione è una scelta di questo modulo. Per la stessa ragione vi risiede il controllo delle chiavi obbligatorie: **quale chiave serva a quale provider è conoscenza del composition root**. L'asimmetria fra le due chiavi è descritta nella sezione 3.2.1.

Due deroghe documentate. Due punti del modulo non discendono da un principio ma da una circostanza.

Il primo: entrambi i provider, quando la chiave manca, rispondono che il servizio è temporaneamente indisponibile anziché segnalare un errore di programmazione, perché una configurazione

incompleta non è un difetto del codice. In un processo avviato regolarmente **quella guardia non può scattare**, perché il controllo delle chiavi obbligatorie avrebbe già impedito l'avvio; resta come difesa in profondità, dato che l'applicazione è costruibile anche senza attraversare la procedura di avvio, come fanno i `test_G` e l'esportazione del contratto dell'`API_G` in `integrazione_G` continua.

Il secondo: il messaggio con cui il controllo delle chiavi rifiuta l'avvio **nomina per esteso la variabile d'ambiente mancante**, l'opposto di quanto il requisito **R-110-F-Ob** impone alle risposte HTTP. La divergenza è voluta, perché quel requisito `G` regola i messaggi rivolti all'utente, mentre il destinatario di questo è chi mette in esercizio il sistema `G` (sezione 3.6.3).

Che cosa è la configurazione e come la si ottiene. Il modulo `app/settings.py` dichiara **che cosa** sia la configurazione: la classe `Settings` elenca l'indirizzo del gateway dei modelli, l'identificativo del modello, le due chiavi di accesso e le origini ammesse dalla politica CORS, ciascuna con il proprio valore predefinito, e le popola dalle variabili d'ambiente. Il modulo `app/dependencies.py` dichiara **come** la si ottenga, perché è compito di un provider. La separazione ha una conseguenza concreta: vincolare `G` la validità della configurazione allo schema che la descrive, imponendo per esempio che una chiave non sia vuota, renderebbe impossibile costruire l'applicazione negli ambienti privi di quelle chiavi, e l'esportazione del contratto dell'`API_G` in `integrazione_G` continua fallirebbe prima di arrivare ai `test_G`.

Firme.

- `get_settings()`: `Settings`: provider memoizzato della configurazione del processo.
- `get_llm_client()`: `LLMClient`: provider memoizzato dell'adattatore del modello linguistico. Il tipo restituito è **la porta**, non il realizzatore.
- `get_content_extractor()`: `ContentExtractor`: provider memoizzato dell'adattatore di estrazione, con la medesima proprietà sul tipo restituito.
- `verifica_chiavi_obbligatorie()`: `None`: solleva un errore di esecuzione, con l'elenco completo delle variabili d'ambiente assenti o vuote, se ne manca una senza la quale il processo non è in grado di servire alcuna richiesta.
- `close_llm_client()`: `None` e `close_content_extractor()`: `None`: rilasciano le risorse dell'adattatore corrispondente, se ne è stato costruito uno.
- `Settings`: attributi `+litellm_base_url: str, +litellm_model: str, +litellm_api_key: str, +tavily_api_key: str, +cors_origins: list[str]`.

3.5 Design pattern adottati

3.5.1 Object Adapter

3.5.2 Facade

3.5.3 Observer

3.5.4 Dependency Injection

3.5.5 Pattern valutati e non adottati

3.6 Idiomi e convenzioni di codifica

Le convenzioni raccolte qui governano la scrittura del codice su entrambi i livelli: vi entra soltanto ciò che vale al di qua e al di là del confine HTTP, mentre quanto riguarda una sola componente

resta nella sezione 3.3 o 3.4. Le tre che seguono hanno in comune il fatto che nessuna delle due componenti potrebbe rispettarle da sola: un flusso si annulla soltanto se ogni anello della catena propaga la chiusura, un contratto è definito una volta sola soltanto se nessuno dei due lati lo riscrive, un errore resta comprensibile all'utente soltanto se ogni punto in cui viene tradotto rispetta lo stesso divieto.

3.6.1 Generatori asincroni e propagazione dell'annullamento

Un testo prodotto nel tempo è rappresentato sui due livelli dallo stesso costrutto, il **generatore asincrono**: `AsyncIterator[str]` nel servizio applicativo, `AsyncIterable<string>` nell'applicazione web. Chi consuma un risultato progressivo lo itera, e non attende che sia completo per riceverlo.

Chi produce un flusso non lo consuma. Un caso d'uso del servizio applicativo restituisce il flusso della porta senza averne prelevato alcun elemento, e la funzione dell'applicazione web che contatta il servizio si comporta allo stesso modo verso il componente che la invoca. Ne segue che **su ciascuno dei due livelli il flusso viene toccato per la prima volta dove un guasto è ancora rappresentabile come esito della richiesta**, e mai più avanti: sul servizio applicativo è l'adattatore di trasporto, che preleva il primo frammento finché lo stato della risposta non è stato inviato (sezione 3.4.2); sull'applicazione web è il controllo sullo stato della risposta, che precede la lettura del corpo e trasforma uno stato negativo in un errore ordinario anziché in un flusso interrotto.

Propagazione dell'annullamento. Il requisito_G **R-79-F-De** (UC71) chiede che un'elaborazione in corso possa essere interrotta. Perché nessuna delle parti coinvolte continui a lavorare, la chiusura deve attraversare quattro passaggi, tutti necessari.

1. L'aggancio che governa il flusso nell'applicazione web possiede un `AbortController` e lo attiva quando l'utente annulla.
2. La funzione che contatta il servizio applicativo passa il segnale alla richiesta HTTP. **Senza questo passaggio l'annullamento sarebbe apparente**: il consumo dei frammenti si fermerebbe, ma la richiesta resterebbe aperta e il servizio continuerebbe a produrre fino al terminatore. È la prima postcondizione di UC71.
3. L'adattatore di trasporto del servizio applicativo verifica a ogni frammento se il client si è disconnesso, e in tal caso interrompe l'invio.
4. La chiusura del generatore restituito dal caso d'uso fa uscire l'adattatore secondario dal contesto della propria richiesta, e la connessione verso il fornitore cade.

Il quarto passaggio avviene senza che alcuno lo scriva: chiudere un generatore asincrono ne interrompe il corpo nel punto in cui era sospeso, e il costrutto che governa la richiesta verso il fornitore rilascia la connessione uscendo. **La propagazione non attraversa la porta come un'operazione dichiarata**, e infatti nessuna delle due porte espone un metodo di chiusura (sezione 3.4.3).

Uscite diverse dall'annullamento. Un consumatore può abbandonare un flusso senza annullarlo: interrompendo il ciclo, restituendo un valore, sollevando un errore a valle. In quei casi il segnale non viene attivato e la catena non parte, quindi la funzione che contatta il servizio applicativo rilascia il lettore del corpo della risposta in un blocco conclusivo, eseguito comunque.

L'eventuale rifiuto di quel rilascio viene ignorato di proposito: su un flusso già annullato il rilascio fallisce, e lasciarlo propagare sostituirebbe l'errore vero in uscita con uno che descrive soltanto la pulizia.

Riconoscimento dell'annullamento. Quando il segnale interrompe la richiesta il browser segnala l'evento con un errore, e distinguerlo da un guasto reale separa un'interfaccia che torna in attesa da una che mostra un messaggio di errore per un'operazione annullata di proposito.

Il riconoscimento avviene **per nome e non per classe**. La classe con cui il browser rappresenta quell'errore appartiene al proprio ambiente di esecuzione, e il confronto per classe fallisce attraversando i confini fra ambienti: riquadri incorporati, processi di servizio e ambiente dei test_G. La conseguenza è misurata: su un annullamento reale il confronto per classe è falso, quindi l'errore cadrebbe nel ramo dei guasti generici e il messaggio tecnico del browser, in lingua inglese, comparirebbe nell'interfaccia contro il requisito_G **R-110-F-Ob**. La stessa convenzione è adottata dal modulo che accede al file system.

Il segnale è un parametro, mai una cattura. La funzione che avvia un'elaborazione riceve il segnale come argomento invece di catturarlo dall'ambiente in cui è stata costruita. La stessa funzione viene conservata per essere rieseguita dal comando che ripete l'ultima operazione, e un segnale catturato alla costruzione sarebbe già attivo al secondo giro: **ripetere un'operazione dopo averla annullata non ripartirebbe mai**.

3.6.2 Tipizzazione dei confini fra i livelli

Le due unità si scambiano dati su HTTP, e la forma di quei dati è l'unica cosa che devono conoscere l'una dell'altra. Il modo in cui quella forma viene stabilita è la convenzione più vincolante_G del prodotto, perché è l'unico punto in cui una divergenza fra i due livelli passerebbe inosservata fino all'esecuzione.

Il contratto si definisce una volta sola e si deriva. La catena parte dagli schemi di validazione del servizio applicativo e arriva ai tipi impiegati dall'applicazione web attraverso tre passaggi automatici: dagli schemi il framework_G web ricava il documento OpenAPI, versionato nella repository_G anziché prodotto al bisogno; da quel documento uno strumento genera le dichiarazioni di tipo; queste vengono riesportate da un unico modulo di raccolta, dal quale il resto dell'applicazione web importa.

Il modulo di raccolta non è un passaggio superfluo: il file generato ha una struttura interna che dipende dal generatore e non dal contratto, quindi importarlo direttamente legherebbe ogni componente a quella struttura e una rigenerazione si propagherebbe ovunque. Con il modulo di raccolta **la rigenerazione si assorbe in un punto solo**. Che il documento OpenAPI sia versionato ha a sua volta due conseguenze pratiche: i tipi si rigenerano senza avere un servizio in esecuzione, e una modifica del contratto compare nel confronto di una *pull request*_G invece di restare invisibile.

Due verifiche_G disgiunte. Sui dati in ingresso agisce la validazione a tempo di esecuzione, che rifiuta un corpo malformato prima che raggiunga il dominio_G. Sui tipi impiegati dall'applicazione web agisce la verifica_G statica, che intercetta in compilazione l'uso di un campo che il contratto non prevede più. Nessuna delle due copre l'altra: la prima controlla *valori* che arrivano da fuori e che nessuna analisi del codice può conoscere in anticipo; la seconda controlla *codice*, e i suoi tipi non sopravvivono alla compilazione.

Un tipo derivabile dallo schema non si scrive a mano. Il divieto riguarda anche i valori, non i soli tipi. Le soglie di lunghezza e il sentinella della correzione grammaticale sono pubblicati dal contratto come valori letterali, e nel modulo di raccolta la costante corrispondente è annotata con il tipo che proviene dal contratto stesso: **cambiare quel valore da un lato solo non compila**. Senza questa convenzione il difetto resterebbe silenzioso, perché un sentinella disallineato non produce un errore ma un'interfaccia che non riconosce più la risposta del servizio.

Le due guardie di verifica_G. I due processi dell'integrazione_G continua descritti nella sezione 2.6 controllano la convenzione ai due capi: quello del servizio applicativo riesporta il documento OpenAPI dal codice e pretende che coincida con il file versionato, quello dell'applicazione web rigenera le dichiarazioni di tipo e pretende che coincidano con quelle versionate. Il primo confronto fallisce quando il contratto pubblicato non corrisponde più al codice che lo produce, il secondo quando il contratto è cambiato senza che il client sia stato adeguato.

3.6.3 Denominazione, struttura dei file e gestione degli errori

Un nome dichiara un ruolo, non una tecnologia. Nessun package di nessuno dei due livelli prende nome da uno strumento o da un fornitore: un package chiamato come una tecnologia attira a sé moduli che con quella tecnologia non hanno rapporto. Sotto la regola stanno alcune corrispondenze fisse: un caso d'uso prende il nome dell'operazione che realizza e vive in un modulo omonimo; i costruttori di dipendenze del servizio applicativo cominciano tutti con lo stesso prefisso, e altrettanto fanno gli agganci dell'applicazione web. Il segno di sottolineatura iniziale marca su entrambi i livelli ciò che un modulo non espone all'esterno: è una convenzione di Python, adottata anche nel codice TypeScript dove il linguaggio non la impone, così che le due basi di codice si leggano allo stesso modo.

Un modulo per unità di responsabilità. La regola si applica anche dove separare costa qualche riga in più. Due casi la illustrano: lo schema che descrive *che cosa* sia la configurazione del processo e il costruttore che dice *come* la si ottenga stanno in moduli distinti, benché insieme farebbero una ventina di righe; la traduzione degli errori di validazione non vive nel modulo che costruisce l'applicazione ma accanto agli schemi di cui conosce i campi. Il criterio è lo stesso nei due casi: **stanno insieme le cose che cambiano per la stessa ragione**.

Due specie di messaggi di errore. La distinzione attraversa entrambi i livelli e vale in ogni punto in cui un errore viene scritto.

- I messaggi **rivolti all'utente** sono in linguaggio naturale italiano, dichiarano causa e azione correttiva e non contengono dettagli tecnici, come impone il requisito_G **R-110-F-Ob**. Comprendono le risposte del servizio applicativo e i pochi messaggi che nascono nell'applicazione web.
- I messaggi **rivolti a chi mette in esercizio il sistema_G** vivono nel registro di esecuzione e nella diagnostica di avvio e seguono la regola opposta: nominano per esteso variabili d'ambiente, operazioni e cause, perché a quel destinatario servono i dettagli che all'altro sono rumore. Il caso limite è il rifiuto di avviare il processo per una chiave mancante (sezione 3.4.5).

Fra le due specie corre un divieto netto: **il testo di un'eccezione non entra mai in un messaggio della prima specie**. Ciò che trapelerebbe non è solo una formulazione tecnica: un errore di validazione non filtrato rimanderebbe al mittente il testo che l'utente ha scritto, un

errore del fornitore riporterebbe nel corpo della risposta la formulazione di un sistema_G esterno in lingua non controllata, un errore del browser comparirebbe in inglese. I tre casi sono riepilogati nella sezione 4.5.

Un errore si converte al confine che lo dichiara. L'ultima convenzione riguarda i tipi e non i messaggi. Ogni tipo di errore è dichiarato accanto al **contratto** che ne prevede la possibilità e non accanto all'implementazione che lo solleva: le eccezioni delle due porte stanno con le porte, quelle di un caso d'uso con il caso d'uso (sezione 3.4.3). Ne discende che **un errore non attraversa mai un confine nella forma in cui è nato**: l'adattatore secondario converte l'eccezione della libreria di trasporto nel tipo dichiarato dalla porta, il caso d'uso converte quella della porta nel proprio, l'adattatore primario converte quella del caso d'uso in uno stato HTTP, l'applicazione web converte lo stato HTTP in uno stato dell'interfaccia. A ogni passaggio l'informazione si riduce, perché quanto serve al passaggio precedente non serve al successivo.

4 Flusso dei dati

4.1 Generazione assistita da LLM_G con risposta in streaming

4.2 Annullamento di un'operazione in corso

4.3 Generazione a partire da un collegamento

Il requisito_G **R-74-F-De** (UC67.2, UC67.5) consente all'utente di indicare l'indirizzo di una pagina web come fonte per la generazione di un testo. Poiché il modello linguistico non accede alla rete, il contenuto della pagina deve essere acquisito dal prodotto e consegnato al modello insieme alle istruzioni.

La presente sezione segue quel percorso dalla richiesta fino alle istruzioni composte, che è la parte specifica di questa funzione, l'unica delle sette a coinvolgere un secondo fornitore esterno. La scelta della sorgente nell'interfaccia è nella sezione 3.3.2; il ritorno del testo generato non differisce dalle altre operazioni assistite ed è nella sezione 4.1.

I tre modi in cui un collegamento può essere rifiutato. Ciascun rifiuto avviene prima di un confine diverso e costa quindi una quantità diversa di lavoro.

1. **Indirizzo malformato**: lo schema della richiesta dichiara il campo come indirizzo web, e un valore che non lo è viene respinto dalla validazione con stato 422. Il rifiuto avviene al confine HTTP e **non raggiunge il dominio_G**.
2. **Indirizzo troppo lungo**: il caso d'uso lo scarta con un errore proprio, che la rotta traduce in stato 400. Il rifiuto avviene nel dominio_G e **non tocca la rete**.
3. **Estrazione fallita**: la pagina è irraggiungibile, inesistente o priva di testo. L'adattatore solleva l'errore dichiarato dalla porta, il caso d'uso lo riconduce al proprio e la rotta lo traduce in stato 503. Il rifiuto avviene dopo la rete e **non raggiunge il modello**.

I tre messaggi che accompagnano questi stati rispettano il requisito_G **R-110-F-Ob** e non riportano il testo delle eccezioni che li hanno originati (sezione 3.6.3).

Ordine di intercettazione dei due errori. L'errore del collegamento scartato è un sottotipo di quello dell'estrazione fallita, così che un chiamante possa trattarli insieme. Nella rotta il sottotipo va perciò intercettato per primo: invertendo i due blocchi, un indirizzo troppo lungo riceverebbe lo stato dell'estrazione fallita e all'utente verrebbe detto che la pagina non è leggibile quando nessuno ha provato a leggerla.

Un limite dell'ordine dei rifiuti. La sequenza descritta vale quando la chiave del servizio di estrazione è configurata. Se manca, il costruttore della dipendenza rifiuta di produrre l'adattatore e, poiché il framework_G web risolve le dipendenze **prima** di validare il corpo della richiesta, ogni chiamata riceve 503, **anche quando l'indirizzo è malformato e meriterebbe un 422**. È una conseguenza dell'ordine di risoluzione e riguarda il solo caso in cui il prodotto sia messo in esercizio senza quella chiave; la differenza rispetto alla chiave del gateway dei modelli, che se manca impedisce l'avvio del processo, è nella sezione 3.2.1.

Dalla richiesta accolta alle istruzioni composte. Superati i controlli, il percorso attraversa i confini descritti nella sezione 3.4 nell'ordine seguente.

1. Il caso d'uso verifica la lunghezza dell'indirizzo e interroga la porta di estrazione.
2. L'adattatore che la realizza inoltra la richiesta al servizio esterno. L'operazione di rete del client di quel servizio è sincrona, quindi viene eseguita su un thread separato per non fermare il ciclo di eventi, secondo la ragione riportata nella sezione 3.4.4.
3. Il contenuto restituito viene troncato dall'adattatore secondo la propria politica, e poi di nuovo dal caso d'uso alla soglia del dominio_G.
4. Il testo estratto viene reso inerte nei suoi marcatori e racchiuso fra i delimitatori, poi collocato nel messaggio destinato al materiale.
5. Il costruttore delle istruzioni compone il messaggio di sistema con le regole proprie di questa funzione, fra cui quelle che dichiarano non autorevole il contenuto ricevuto.
6. Il caso d'uso interroga la porta del modello linguistico e restituisce il flusso, che da questo punto in avanti si comporta come quello di ogni altra operazione assistita.

Il troncamento avviene due volte. Le due soglie coincidono nel valore, il che rende la doppia applicazione facile da scambiare per una svista. La ragione sta in ciò che la porta dichiara: la porta di estrazione **non promette alcun limite di lunghezza** sul risultato, quindi il troncamento dell'adattatore è una sua politica, che un realizzatore diverso o un doppio predisposto per un test_G non sarebbero tenuti a rispettare. Il caso d'uso applica la propria soglia perché è la sola di cui risponde: se le due divergessero, il dominio_G resterebbe comunque entro il limite che dichiara.

Dove interviene ciascuna difesa sul contenuto di terze parti. Le tre difese contro le istruzioni ostili contenute in una pagina, e il loro limite, sono descritte nella sezione 3.4.3; qui rileva il punto della sequenza in cui ciascuna interviene, perché è l'ordine a renderle efficaci. La separazione dei ruoli agisce al passo 4, collocando il contenuto nel messaggio del materiale e mai fra le istruzioni. La neutralizzazione dei delimitatori agisce nello stesso passo e **prima** che il testo sia racchiuso, altrimenti una pagina potrebbe chiudere il recinto per conto proprio; avviene inoltre **dopo** il troncamento del passo 3 e sostituisce i caratteri senza aggiungerne, perché allungare il testo farebbe superare alle istruzioni un limite che il dominio_G considera rispettato.

La dichiarazione di non autorevolezza agisce al passo 5, nel messaggio di sistema, l'unico luogo che la pagina non può raggiungere.

L'estrazione precede l'inizio del flusso. Questo è l'unico dei sette casi d'uso dichiarato `async def` (sezione 3.4.3), e il flusso appena descritto è il luogo in cui il motivo si vede. Se fosse scritto come generatore, il corpo resterebbe fermo finché nessuno ne consuma il risultato: i passi da 1 a 5 avverrebbero dentro l'adattatore di trasporto, cioè dopo l'invio dello stato della risposta. **Nessuno dei tre stati di rifiuto sarebbe più possibile**, e un collegamento troppo lungo o una pagina irraggiungibile si presenterebbero all'utente come un flusso interrotto invece che come un errore. Con la forma adottata l'estrazione avviene subito e ciò che viene restituito resta un flusso non consumato, la proprietà che la sezione 3.6.1 chiede a entrambi i livelli.

4.4 Salvataggio e caricamento di una nota

4.5 Propagazione e presentazione degli errori

Le sezioni precedenti seguono le interazioni che riescono. Questa segue quelle che falliscono, dal punto in cui il guasto si manifesta fino alla frase che l'utente legge. I requisiti_G coinvolti sono **R-110-F-Ob**, che impone messaggi in linguaggio naturale privi di dettagli tecnici, e **R-80-F-Ob** (UC72), che chiede di avvisare l'utente quando un'operazione non si completa.

Il momento del guasto determina la forma della risposta. Il medesimo guasto del gateway dei modelli produce due esiti diversi a seconda del momento in cui si manifesta, e il momento che li separa è l'invio dell'intestazione di stato. Prima di quell'istante lo stato è ancora modificabile e il guasto diventa una risposta di errore ordinaria; dopo, lo stato inviato è già quello di una risposta riuscita e non è ritrattabile, quindi l'unica via che resta è dichiarare il fallimento dentro il flusso. **La stessa causa produce così due rappresentazioni diverse**, non per scelta ma perché il protocollo non consente altro. È anche la ragione per cui l'adattatore di trasporto preleva il primo frammento prima di restituire la risposta (sezioni 3.4.2 e 3.6.1): quel prelievo sposta il maggior numero possibile di guasti dalla seconda categoria alla prima.

Prima dell'intestazione: una forma sola per ogni errore. Tutte le risposte di errore del servizio applicativo hanno la stessa forma, un solo campo testuale destinato a essere mostrato. Vi confluiscono esiti di origine diversa: la validazione della richiesta, che compone la propria frase a partire dal campo che non ha superato il controllo (stato 422); i rifiuti propri di una singola operazione, come quelli della sezione 4.3 (stati 400 e 503); l'indisponibilità del gateway all'apertura del flusso (stato 503, UC72). L'uniformità consente all'applicazione web di leggere un campo solo e al contratto di dichiarare una forma sola, così che i tipi generati non descrivano risposte che il servizio non produce.

Dopo l'intestazione: l'evento terminale d'errore. Quando il guasto sopraggiunge a flusso già avviato, il servizio applicativo emette un evento terminale che dichiara il fallimento e registra l'accaduto nel proprio registro di esecuzione; il formato di quell'evento è nella sezione 4.1.

Senza un evento dedicato un flusso interrotto e uno completato sarebbero **indistinguibili per il client**, che in entrambi i casi osserverebbe soltanto la chiusura della connessione: l'utente riceverebbe un testo mozzo presentato come completo, che è la condizione vietata da **R-80-F-Ob**. La distinzione è quindi ottenuta per affermazione e non per implicazione.

I tre esiti che il client distingue. Il consumo di una risposta progressiva termina in uno di tre modi, e l'applicazione web li tratta diversamente.

- **Terminatore di completamento:** l'operazione è riuscita, il testo accumulato è integro.
- **Evento terminale d'errore:** l'operazione è fallita dopo essere cominciata. Il testo già arrivato resta visibile, ma accanto compare il messaggio che ne dichiara l'incompletezza.
- **Chiusura senza alcun terminatore:** la connessione è caduta e il servizio non ha detto nulla. Questo è l'unico caso in cui il messaggio d'errore **nasce nell'applicazione web**, perché è l'unico in cui non ne esiste uno proveniente dal servizio. Anche quel messaggio dichiara causa e azione correttiva senza dettagli tecnici, come tutti gli altri.

Il terzo caso è il più insidioso, perché una connessione che cade è indistinguibile da un completamento se non si è deciso in anticipo che il completamento debba essere dichiarato.

Il riconoscimento avviene sul nome del campo, non sul testo della riga. Riconoscere l'evento d'errore confrontando il testo della riga con una stringa nota funzionerebbe fino al giorno in cui il modello linguistico producesse quella stessa stringa dentro il testo generato: un frammento del contenuto verrebbe scambiato per la dichiarazione di un fallimento e l'operazione risulterebbe interrotta senza esserlo. Su un prodotto che genera Markdown arbitrario, fra le cui funzioni c'è la scrittura di blocchi di codice, la collisione è verosimile; il riconoscimento per nome del campo la rende impossibile per costruzione.

Dal punto in cui nasce a quello in cui l'utente lo legge. Il percorso di un messaggio d'errore attraversa quattro passaggi, e nessuno lo riformula.

1. Il servizio applicativo lo produce, nella forma unica descritta sopra oppure dentro l'evento terminale.
2. La funzione dell'applicazione web che contatta il servizio ne estrae il testo e lo solleva come errore ordinario, unificando i due casi: da qui in avanti un fallimento prima e uno dopo l'intestazione sono la stessa cosa.
3. Lo stato condiviso registra il messaggio e spegne insieme l'indicatore di avanzamento. I due effetti sono deliberatamente inseparabili: un errore a video con l'indicatore ancora acceso descriverebbe una situazione che non esiste, un'operazione insieme fallita e in corso.
4. L'interfaccia lo presenta in un riquadro di avviso collocato in un'area annunciata dai lettori di schermo, così che l'esito raggiunga anche chi non guarda lo schermo.

Un caso resta fuori da questo percorso: l'annullamento volontario. Il browser lo segnala con un errore, ma un'operazione interrotta di proposito non è un fallimento e non produce alcun messaggio; il riconoscimento che lo distingue è nella sezione 3.6.1.

Che cosa non trapela, nei tre punti in cui potrebbe. Il requisito_G **R-110-F-Ob** vieta i dettagli tecnici nei messaggi rivolti all'utente. Il divieto è rispettato in tre punti, ciascuno dei quali trattiene qualcosa di diverso.

- **Validazione della richiesta:** la forma predefinita del framework_G web espone il tipo dell'errore, la posizione del campo e il contesto del controllo, e rimanda al mittente il valore ricevuto, cioè **il testo che l'utente ha scritto**. Nessuno dei quattro raggiunge la risposta.

- **Guasto del fornitore:** la formulazione dell'eccezione, prodotta da un sistema_G esterno e in lingua non controllata, resta nel registro di esecuzione; nel corpo della risposta va un messaggio del prodotto.
- **Annullamento:** il testo con cui il browser segnala l'interruzione è tecnico e in lingua inglese, e non raggiunge l'interfaccia.

Nei tre casi l'informazione utile alla diagnosi è conservata dove serve a chi mette in esercizio il sistema_G e sostituita da una frase del prodotto dove il destinatario è l'utente, secondo la distinzione fra le due specie di messaggi stabilita nella sezione 3.6.3.

5 Tracciamento dei requisiti_G

5.1 Criteri di tracciamento

5.2 Requisiti_G funzionali

5.3 Requisiti_G di qualità

5.4 Requisiti_G di vincolo_G

5.5 Grafici riassuntivi